

（1）Cortex-M3/M4程序设计

编程语言：C语言

基本开发方式：

（1）寄存器编程：（直接读写外设寄存器）

优点：代码最精简

缺点：寄存器难以记忆，移植性差

（2）库函数编程：（采用标准封装库CMSIS）

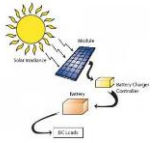
优点：编程方便，移植性好

缺点：代码不够精简，运行速度略差

（3）可视化界面自动生成代码：（CubeMX的HAL/LL库）

优点：可视化配置，入门方便，编程快捷

缺点：代码不够精简，难以实现较复杂的功能



（1）Cortex-M3/M4程序设计

寄存器编程与库函数编程比较

例：将GPIOF.0设置为高电平

（1）寄存器编程：（直接读写外设寄存器）

```
GPIOF->BSRRL=0x0001
```

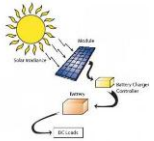
（2）库函数编程：（采用标准封装库CMSIS）

```
GPIO_SetBits(GPIOF, GPIO_Pin_0);
```

```
Void GPIO_SetBits(GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
{
    GPIOx->BSRRL=GPIO_Pin;
}
```

（3）Cube编程：（LL库）

```
LL_GPIO_SetOutputPin(GPIOF, LL_GPIO_Pin_0);
```



（1）Cortex-M3/M4程序设计

CMSIS (Cortex microcontroller software interface standard):
包含了微控制器的外设驱动库、嵌入式操作系统以及协议栈等第三方软件。

优点：

- 易于学习，降低了系统开发难度；
- 提高了软件的可重用性；
- 提高软件的兼任性

常用包括：

- **CMSIS内核和外设驱动；**
- **CMSIS-DSP库**
- **CMSIS-RTOS**

开发环境和流程

μVision (Keil)是目前最广泛应用的Cortex开发平台

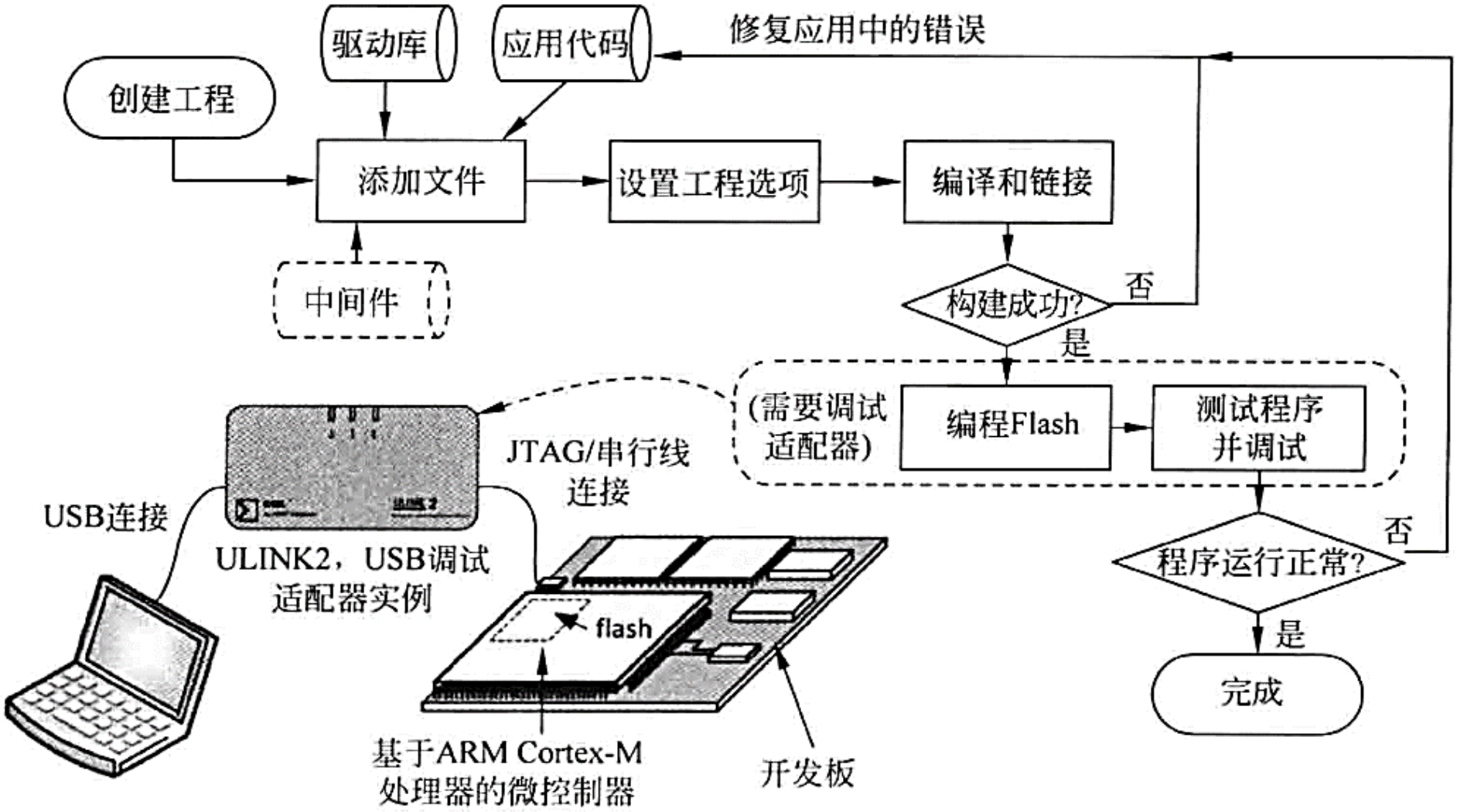
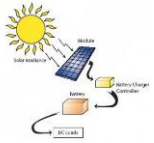


图 2.4　简化的软件开发流程



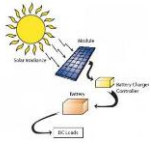
（2）C语言相关知识

2.1 变量定义

Cortex变量类型	标准C中变量类型
uint8_t	unsigned char
uint16_t	unsigned short int
uint32_t	unsigned int
int8_t	signed char
int16_t	signed char
int32_t	signed int

uint8_t 等在标准库的stdint.h文件中重定义：

```
typedef  unsigned  char  uint8_t;
```



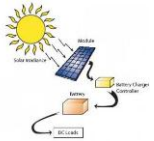
（2） C语言相关知识

2.2 include 语句

#include 常用来包含某个头文件。
头文件分为系统头文件和自定义头文件

系统头文件以<>为标识，编译器在编译时，会在编译系统安装目录的库文件夹中搜寻该文件。
例如: **#include <stdio.h>**

自定义头文件以“ ”为标识，编译器在编译时，首先在用户的文件目录中搜寻该文件，如果找不到，则会跳到系统库文件目录中继续搜寻。
例如: **#include “config.h”**。
自定义头文件一般需要在编译器中设置包含目录。



（2） C语言相关知识

2.3 预处理语句

➢ 预处理语句以“#”开头，是给编译器传达指示。

1. 预处理全局变量，例

```
#define GPIOH_MODER    *(unsigned int*)(GPIOH_BASE+0x00)
```

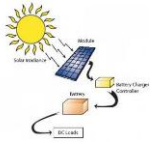
2. 预处理常量

```
#define   TRUE    1
```

#undef

有时候我们想要定义一个宏，但是名字又非常普通（如MAX_NUM），担心跟系统中的其他宏定义相冲突，这个时候我们可以使用 #undef来解除该宏定义。如：

```
#undef MAX_NUM
```



（2） C语言相关知识

2.4 条件编译语句

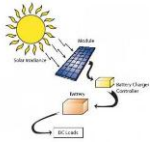
- #ifdef 标识符
  程序段1
  #else
  程序段2
  #endif

作用：
“标识符”已经被定义时，编译程序段1进行，否则编译程序段2。

例：头文件中经常包含以下代码

```
#ifndef __GPIO_H
#define __GPIO_H
        ...
#endif
```

使用#ifndef的目的是为了防止重复包含。在大型程序中，为了防止出现一个头文件被同一个.C文件或头文件包含两次及以上，需要采用如下结构来保证该头文件只会被包含一次。



（2） C语言相关知识

2.5 extern 外部声明

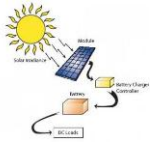
在C语言中，extern表明变量或函数是定义在其他文件中的。在一个文件中定义全局变量，而在其他文件中用extern对全局变量声明。如

```
extern uint8_t value1;

extern uint16_t p(void);
```

编译（Compiling）与链接（Linking）对比

阶段	处理工具	核心任务	典型错误
编译（Compiling）	编译器 (Compiler)	语法检查、将 C 代码翻译成特定架构（如 Cortex-M4）的**汇编/机器码**。	语法错误、变量未定义、括号不匹配。
链接（Linking）	链接器 (Linker)	将多个 .o 文件及库文件合并，**分配地址**，解析跨文件的符号引用。	符号重复定义、Undefined reference (找不到 extern 变量)。



（2） C语言相关知识

2.6 位运算操作

C语言位运算操作符有6个

- & 按位与
- | 按位或
- ^ 按位异或
- ~ 按位非
- << 左移
- >> 右移

例：

```
GPIOH_MODER&= ~(0x03<<20);
/*将GPIOH_MODER的第20、21位清零 */
GPIOH_MODER |= (0x01<<20);
/*将GPIOH_MODER的第20位设置为1 */
```

（2） C语言相关知识

2.7 __IO 关键字

__IO 关键字的含义

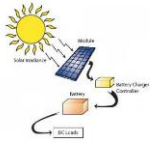
在标准库的core_cm4.h文件中定义：

#define __IO volatile

- volatile变量允许程序之外的硬件或软件修改其内容 。
- 确保当前读取到的值是最新的数据值，防止编译器优化代码后以寄存器值代替实际值。

例：

```
__IO uint32_t tmp = 0x00010000;
......
 tmp = GPIOx->LCKR;              /* Read LCKR bit*/
 tmp = GPIOx->LCKR;
……
```



（2） C语言相关知识

2.8 结构体与外设定义

结构体的定义将不同的数据类型整合为一个有机整体。

例：GPIO口定义

```
typedef struct
{
  __IO uint32_t MODER;
  __IO uint32_t OTYPER;
  __IO uint32_t OSPEEDR;
  __IO uint32_t PUPDR;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint16_t BSRRL
  __IO uint16_t BSRRH
  __IO uint32_t LCKR
  __IO uint32_t AFR[2];   /*Address offset: 0x20-0x24 */
} GPIO_TypeDef;
```

注意：
结构体成员在内存中的地址按顺序排列！

结构体变量的地址对应关系

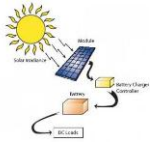
用结构体定义外设的优点：

- 相关的设置参数放置在一个结构体中；
- 对外设操作时，将相关寄存器地址映射到结构体的成员。
- 结构体变量地址对应该结构体的首地址

```
typedef struct
{
  __IO uint32_t MODER;
  __IO uint32_t OTYPER;
  __IO uint32_t OSPEEDR;
  __IO uint32_t PUPDR;
  __IO uint32_t IDR;
```

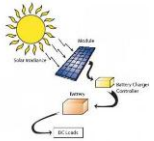
GPIO 寄存器映射

偏移	寄存器	31	30	29	28
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]	
	Reset value	1	0	1	0
0x04	GPIOx_OTYPER (where x = A..I/)				
	Reset value				
0x08	GPIOx_OSPEEDER (where x = A..I/ except B)	OSPEEDR15[1:0]		OSPEEDR14[1:0]	
	Reset value	0	0	0	0
0x0C	GPIOA_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]	
	Reset value	0	1	1	0



GPIO存贮器地址映射表

地址	GPIO
0x4002 2000- 0x4002 23FF	GPIOI
0x4002 1C00- 0x4002 1FFF	GPIOH
0x4002 1800- 0x4002 1BFF	GPIOG
0x4002 1400- 0x4002 17FF	GPIOF
0x4002 1000- 0x4002 13FF	GPIOE
0x4002 0C00- 0x4002 0FFF	GPIOD
0x4002 0800- 0x4002 0BFF	GPIOC
0x4002 0400- 0x4002 07FF	GPIOB
0x4002 0000- 0x4002 03FF	GPIOA



预定义外设地址

```
#define GPIOB       ((GPIO_TypeDef *) GPIOB_BASE)
```

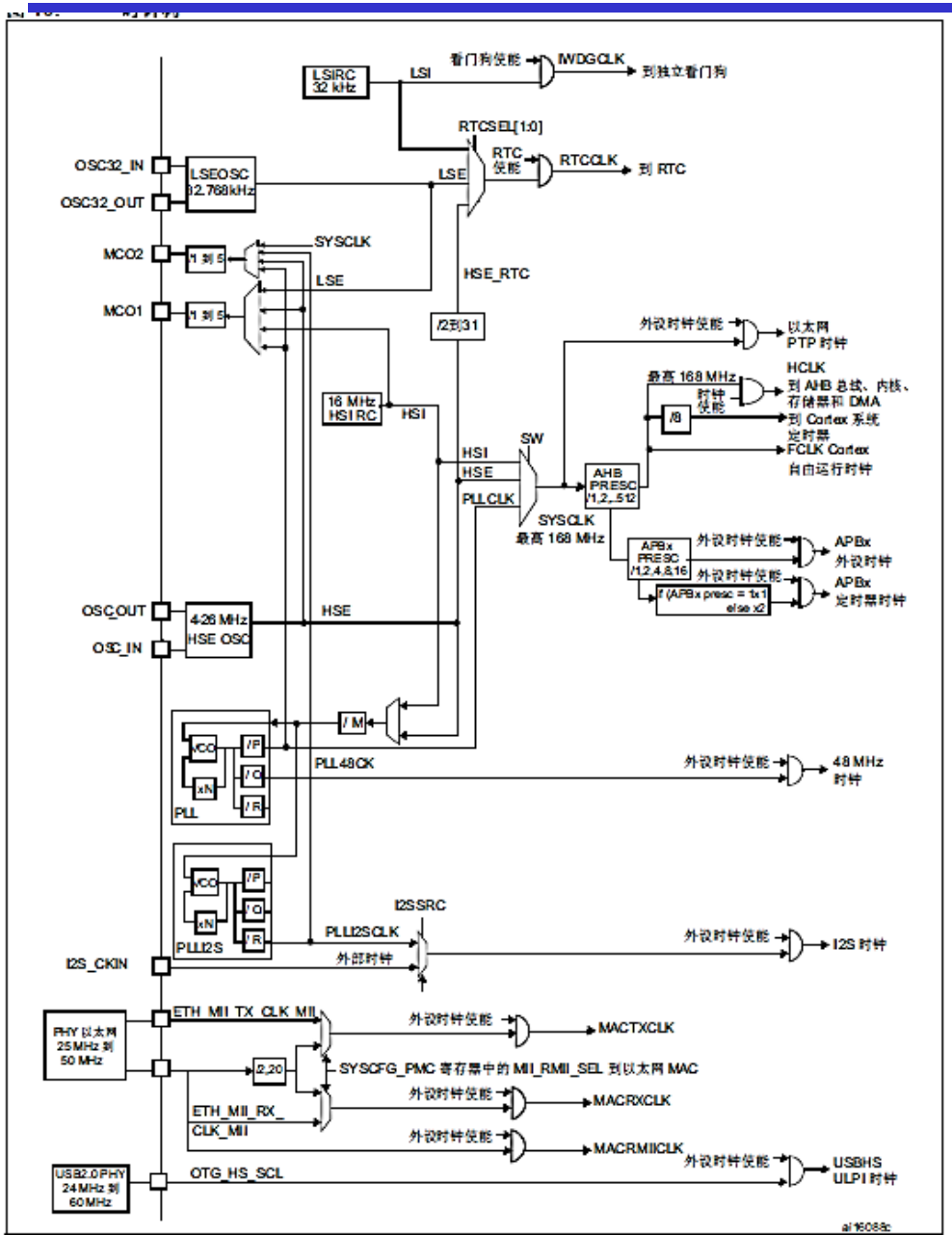
```
#define GPIOB_BASE     (AHB1PERIPH_BASE + 0x0400)

#define AHB1PERIPH_BASE  (PERIPH_BASE + 0x00020000)

#define PERIPH_BASE          ((uint32_t)0x40000000)
```

将GPIOB定义为指向结构体的指针，其值为0x4002 0400
该结构体成员的地址按长度依次排列，
如：GPIOB-> OTYPER 的地址为0x4002 0404

（3）Cortex的时钟树和时钟设置



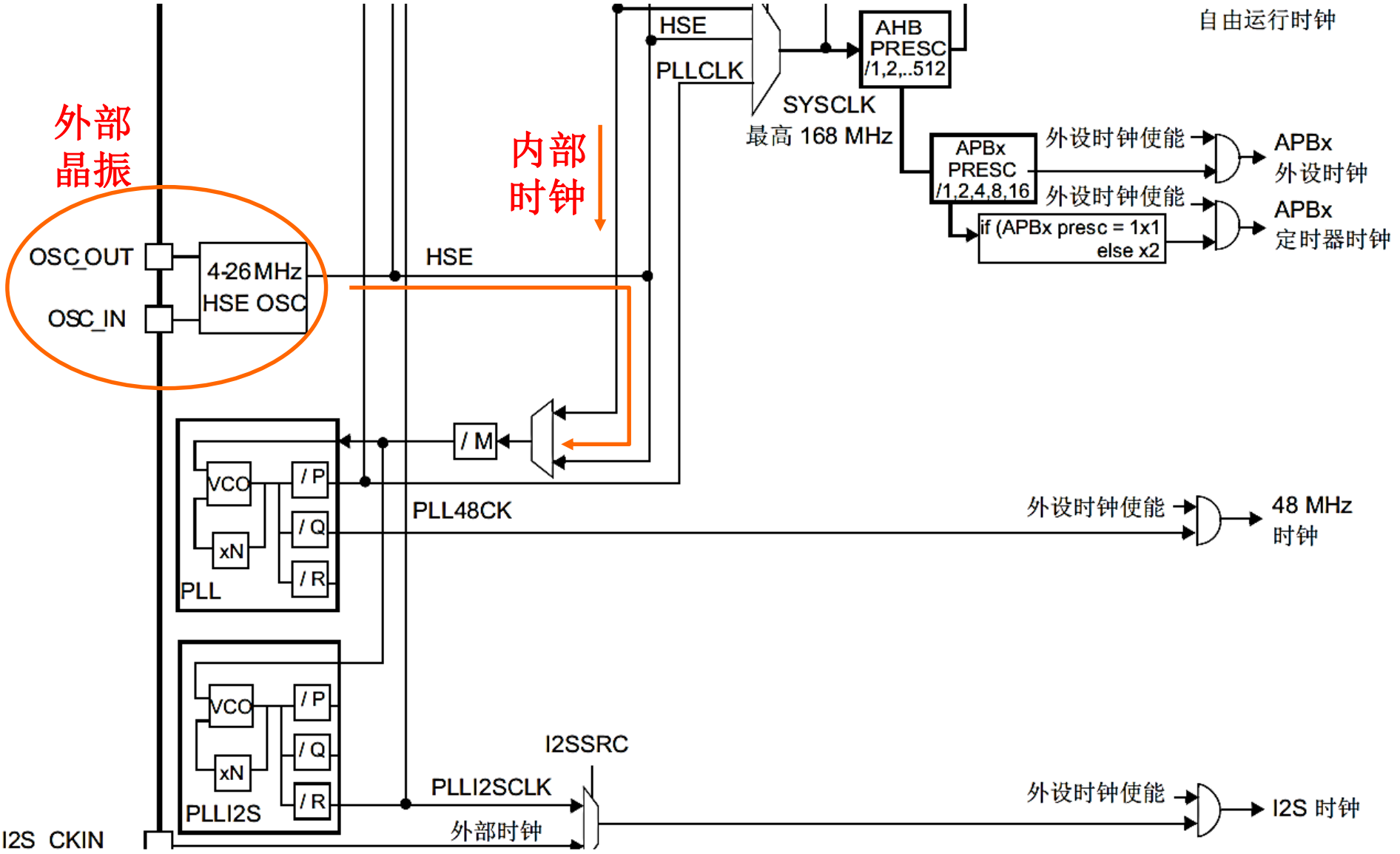
时钟树特点：

1、多个时钟源；

2、可灵活设置工作频率；

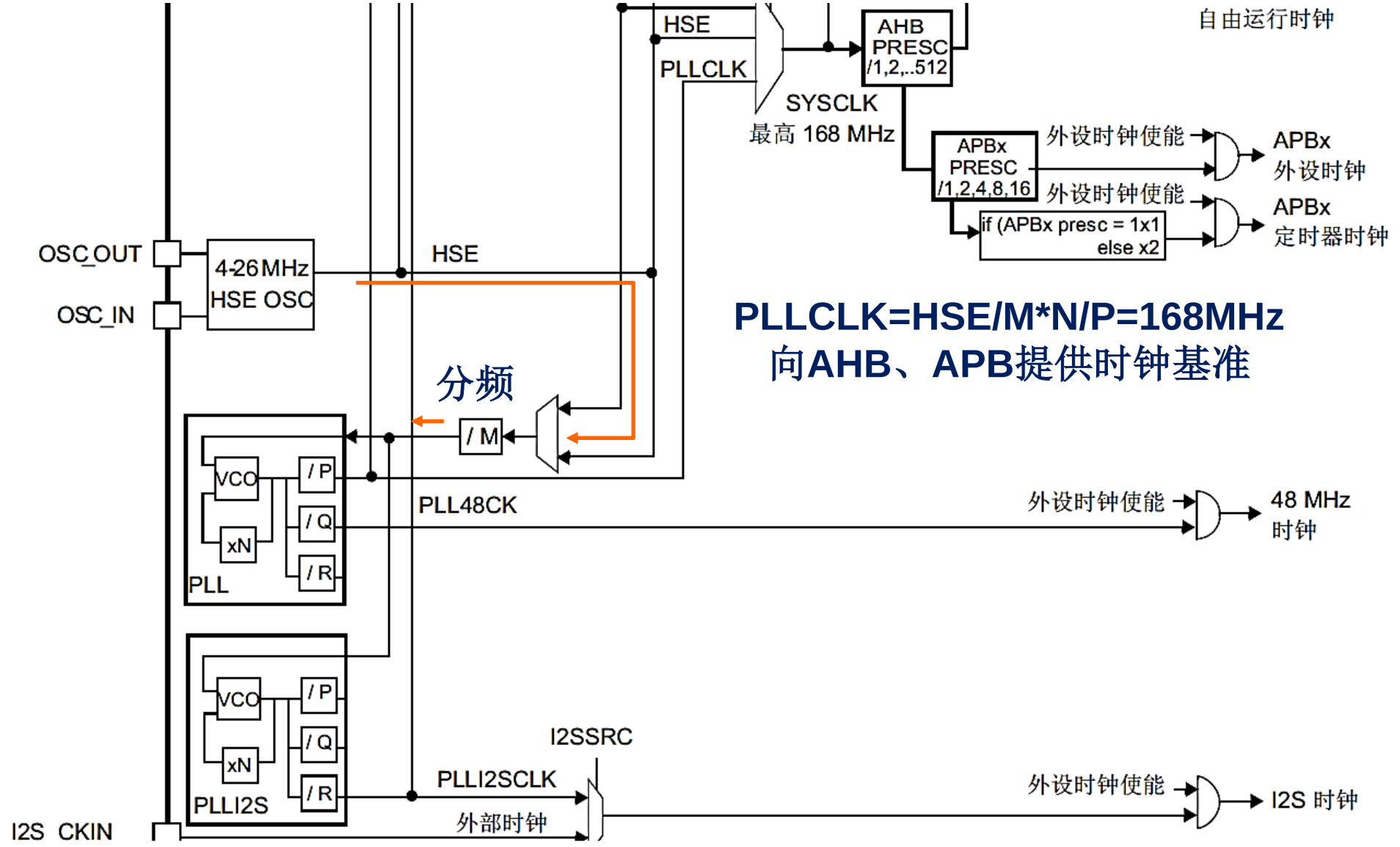
时钟树和时钟设置

主时钟的产生



时钟树和时钟设置

主时钟的产生

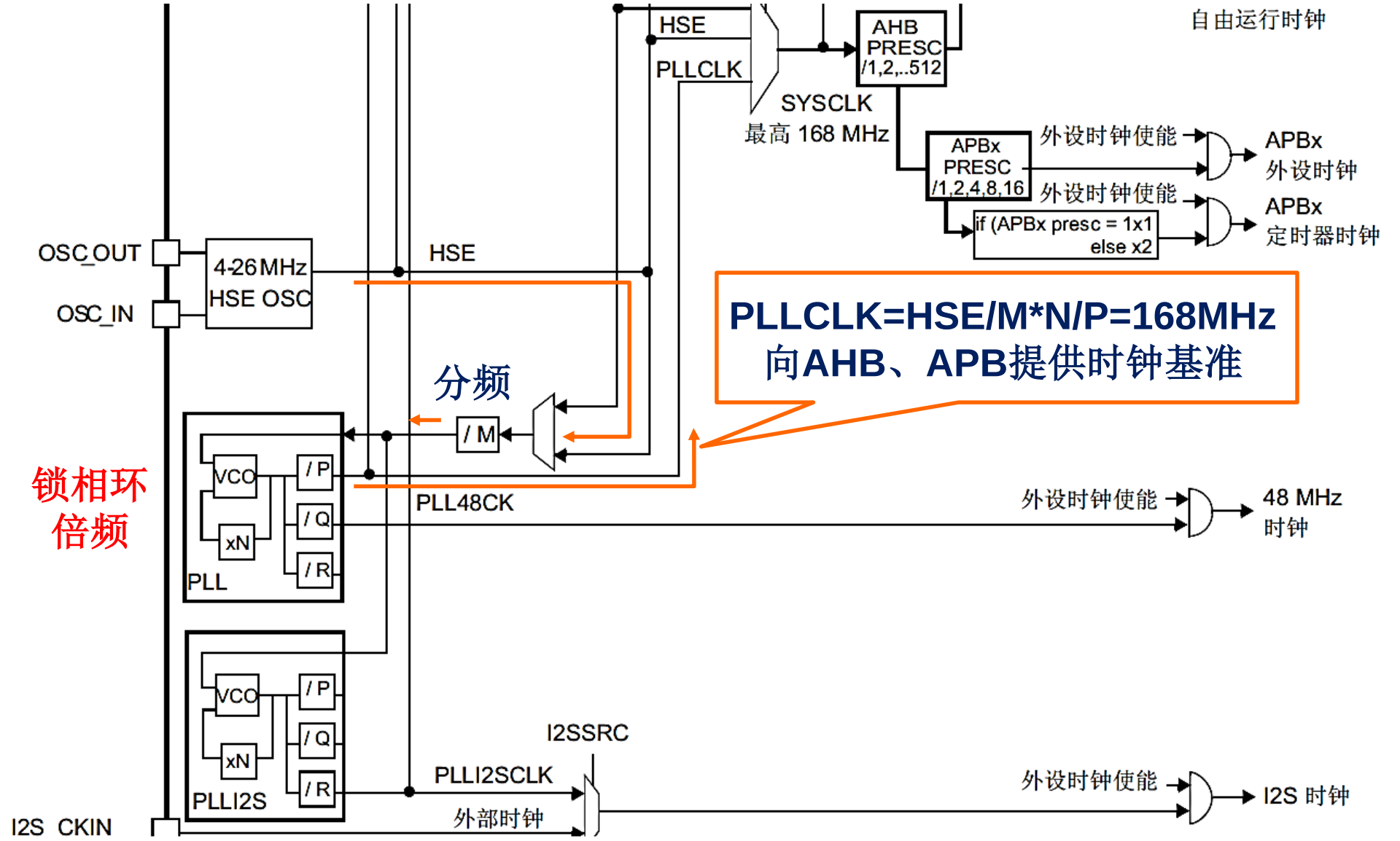


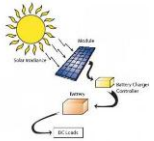
PLLCLK=HSE/M*N/P=168MHz

向AHB、APB提供时钟基准

时钟树和时钟设置

主时钟的产生





系统初始化和时钟初始化

在keil的MDK开发下环境下，系统自动加载启动文件startup_STM32F40_41xxx.S，复位代码

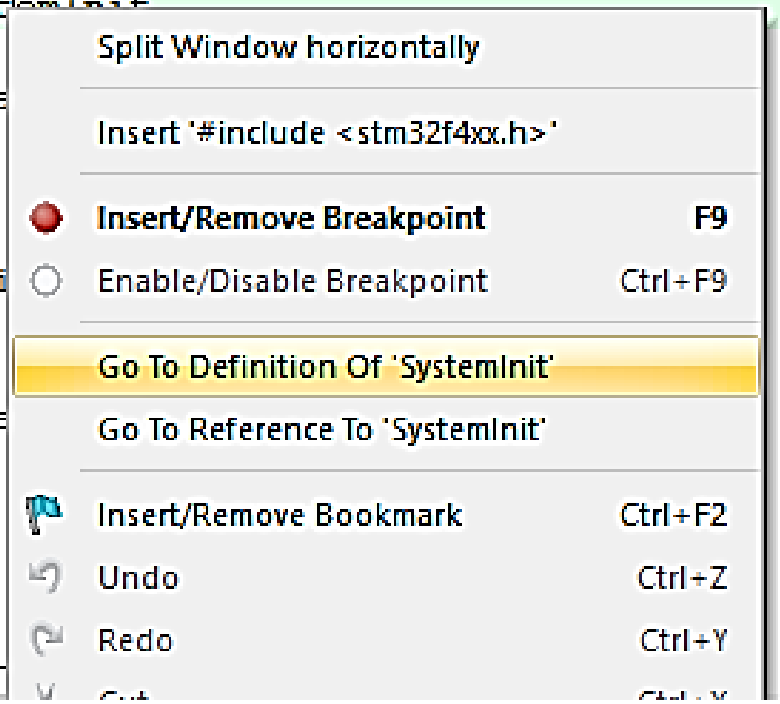
```
; Reset handler
Reset_Handler    PROC                    //表示程序开始
        EXPORT  Reset_Handler    [WEAK]
        IMPORT  SystemInit               //该变量在其他文件定义
        IMPORT  __main

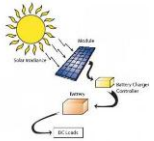
                LDR     R0, =SystemInit
                BLX     R0
                LDR     R0, =__main
                BX      R0
                ENDP
```

系统初始化和时钟初始化

光标停留在 **SystemInit**，点击鼠标右键，弹出如下菜单；
点击**Go To Definition Of ‘SystemInit’**

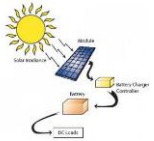
```
7 Reset_Handler     PROC
8                   EXPORT  Reset_Handler              [WEAK]
9         IMPORT  SystemInit
0         IMPORT  __main
1
2                   LDR     R0, =Syst
3                   BLX     R0
4                   LDR     R0, =__ma
5                   BX      R0
6                   ENDP
7
8 ; Dummy Exception Handlers (infini
9
0 NMI_Handler       PROC
1                   EXPORT  NMI_Handle
2                   B       .
3                   ENDP
4 HardFault_Handler\
5                   PROC
6                   EXPORT  HardFault_
7                   B
```





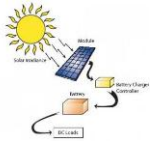
系统初始化和时钟初始化

```
void SystemInit(void)
{
  /* FPU settings ------------------------------------------------------
-*/
  #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
    SCB->CPACR |= ((3UL << 10*2)|(3UL << 11*2));  /* set
CP10 and CP11 Full Access */
  #endif
……
AHB/APBx prescalers and Flash settings ----------------*/
 SetSysClock();
……
  /* Configure the Vector Table location add offset address
-----------------*/
}
```



系统初始化和时钟初始化

```
static void SetSysClock(void)
{
  __IO uint32_t StartUpCounter = 0, HSEStatus = 0;
……
  /* Enable HSE */
  RCC->CR |= ((uint32_t)RCC_CR_HSEON);
……
    /* Configure the main PLL */
RCC->PLLCFGR = PLL_M | (PLL_N << 6) | (((PLL_P >> 1) -1)
<< 16) |(RCC_PLLCFGR_PLLSRC_HSE) | (PLL_Q << 24);
    /* Enable the main PLL */
    RCC->CR |= RCC_CR_PLLON;
……
}
```



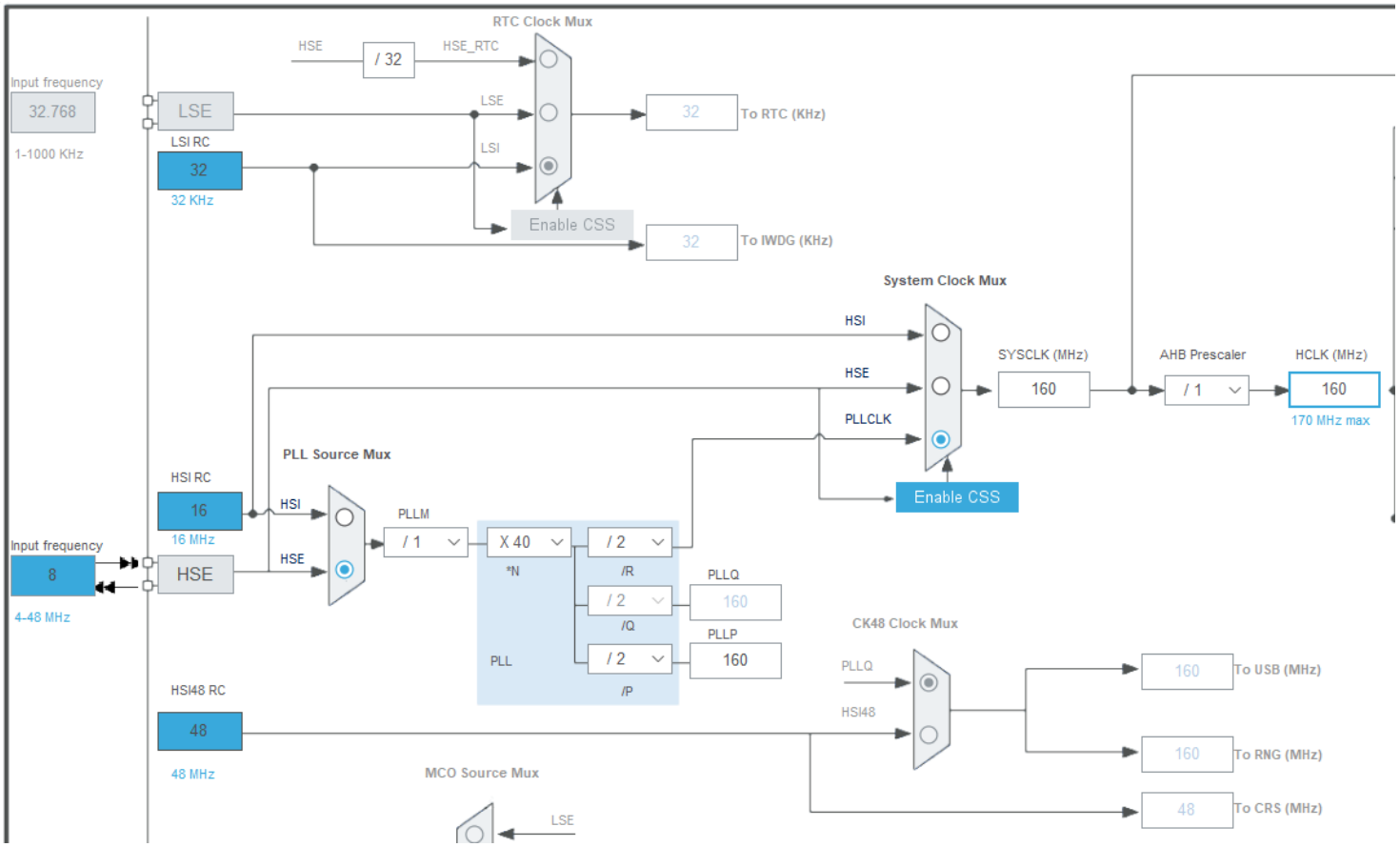
系统初始化和时钟初始化

PLL_M, PLL_N, PLL_P 在 stm32f4xx.h中定义

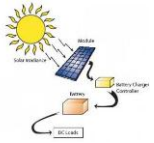
```
#define PLL_M      8
#define PLL_N      336
#define PLL_P      2
```

晶振8MHz，PLLCLK=8M/8*336/2=168MHz

可视化软件Cube MX时钟配置



使用**Cube MX**软件可以通过以上界面配置时钟，自动生成代码。



（4）通用IO口

16位 GPIOA　16位 GPIOB　16位 GPIOC　……　GPIOI

配置寄存器

寄存器名	功能
GPIOx_MODER	IO口类型设置
GPIOx_OTYPER	输出类型设置
OSPEEDER	输出速度
GPIOx_PUPDR	上拉或下拉设置
GPIOx_IDR, GPIOx_ODR	输入或输出数据
GPIOx_BSRR	位清零或置1
GPIOx_AFRH, GPIOx_AFRL	复用功能选择

GPIO模式寄存器

GPIO 端口模式寄存器 (GPIOx_MODER) (x = A..I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y:2y+1 **MODERy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入，用于配置 I/O 方向模式。

00：输入（复位状态）

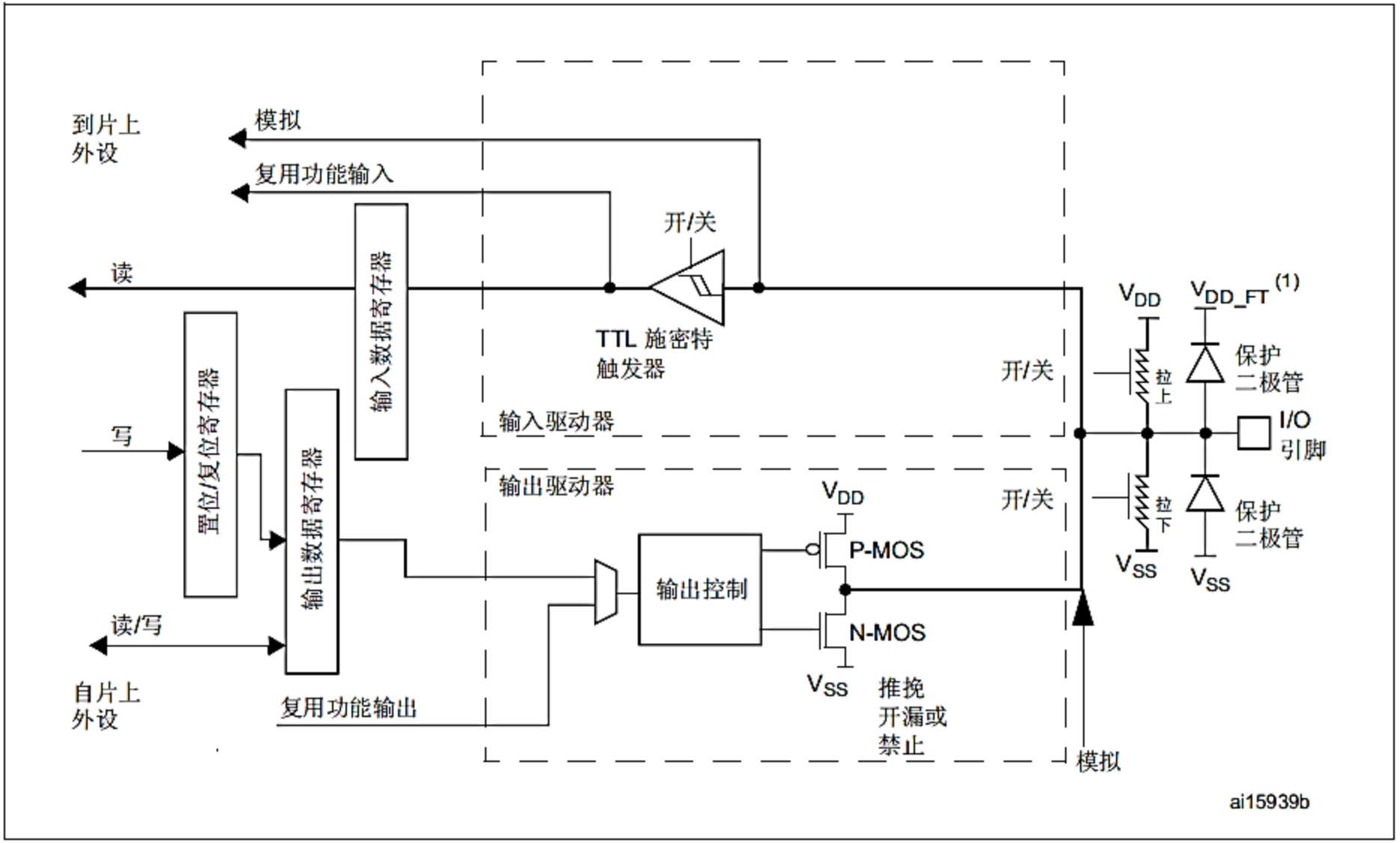
01：通用输出模式

10：复用功能模式

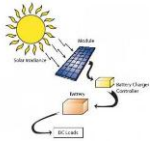
11：模拟模式

GPIO结构

图 17.　　5 V 容忍 I/O 端口位的基本结构



1. V_{DD_FT} 是和 5 V 容忍 I/O 相关的电位，与 V_{DD} 不同。



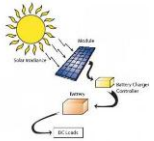
通用IO口初始化程序

采用库函数编程：

```
GPIO_InitTypeDef GPIO_InitStructure;  //定义结构类型
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

结构体定义：

```
typedef struct
{
  uint32_t GPIO_Pin;
  GPIOMode_TypeDef GPIO_Mode;
  GPIOSpeed_TypeDef GPIO_Speed;
  GPIOOType_TypeDef GPIO_OType;
  GPIOPuPd_TypeDef GPIO_PuPd;
  GPIOPuPd_TypeDef */
}GPIO_InitTypeDef;
```



通用IO口操作

采用库函数编程：

```
GPIO_ResetBits(GPIOA , GPIO_Pin_9);     //对GPIOA.9清零
GPIO_SetBits(GPIOA , GPIO_Pin_9);       //对GPIOA.9置1
```

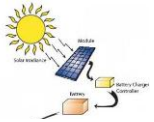
等效的直接对寄存器操作：

```
GPIOA->BSRRH= GPIO_Pin_9;       //对GPIOA.9清零
GPIOA->BSRRL= GPIO_Pin_9;       //对GPIOA.9置1
```

```
//GPIO_Pin_9预定义
#define GPIO_Pin_9                ((uint16_t)0x0200)
```

```
GPIOA->BSRRL= GPIO_Pin_9| GPIO_Pin_10 ;
//对GPIOA.9和GPIOA.10置1
```

设置GPIO口为什么不通过ODR寄存器操作，而用BSRRH/BSRRL寄存器？



GPIO 端口置位/复位寄存器 (GPIOx_BSRR) (x = A..I)

GPIO port bit set/reset register

偏移地址：0x18

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 **BRy**：端口 x 复位位 y (Port x reset bit y) (y = 0..15)

这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回值 0x0000。

0：不会对相应的 ODRx 位执行任何操作

1：对相应的 ODRx 位进行复位

注意：如果同时对 BSx 和 BRx 置位，则 BSx 的优先级更高。

位 15:0 **BSy**：端口 x 置位位 y (Port x set bit y) (y= 0..15)

这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回值 0x0000。

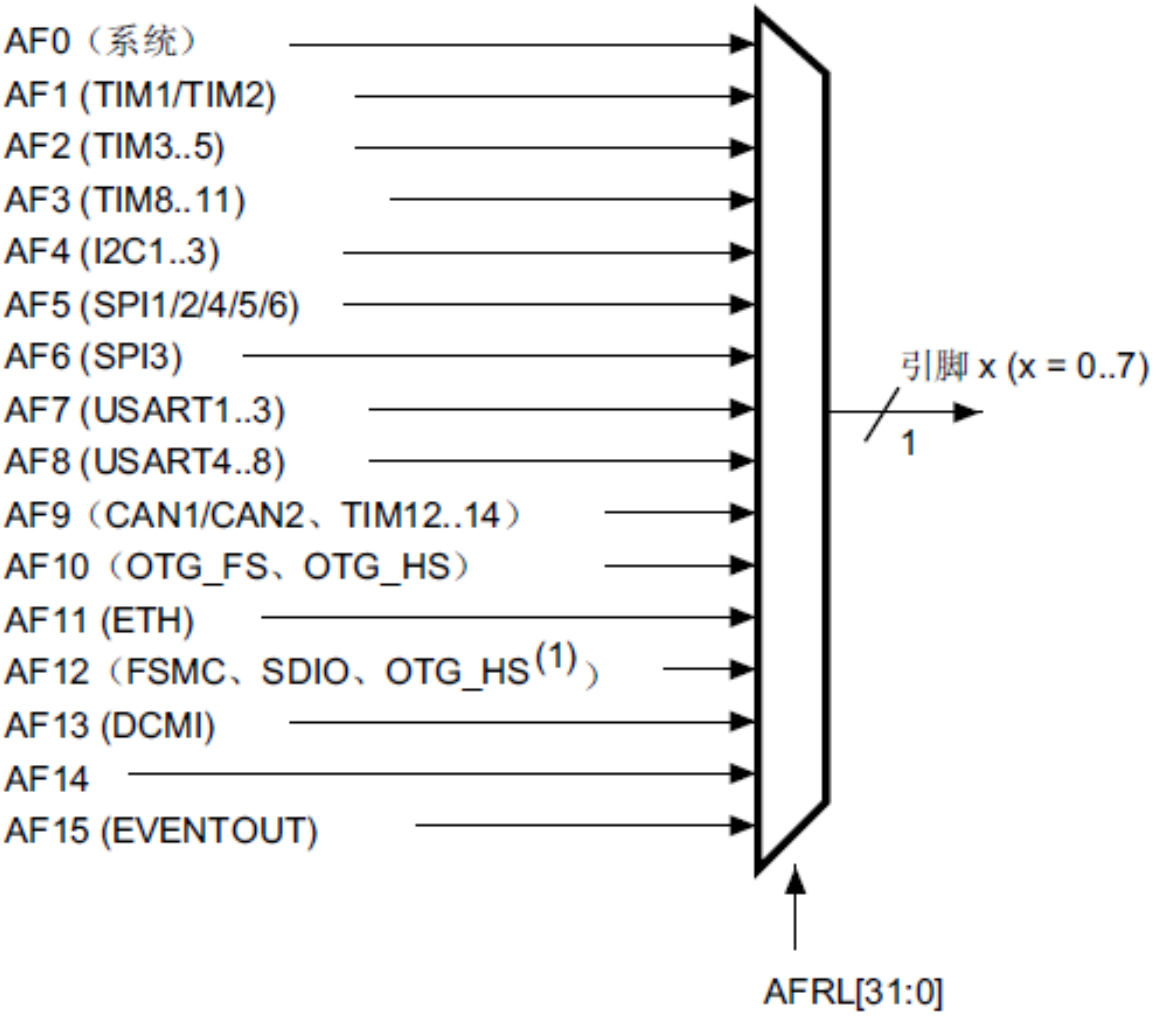
0：不会对相应的 ODRx 位执行任何操作

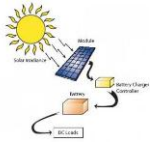
1：对相应的 ODRx 位进行置位

通用IO口操作

GPIO的复用功能

对于引脚 0 到引脚 7GPIOx_AFRL[31:0] 寄存器会选择专用的复用功能



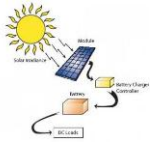


STM32F407的IO口复用功能表

Table 8. Alternate function mapping (continued)

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6
		SYS	TIM1/2	TIM3/4/5	TIM8/9/10/11	I2C1/2/3	SPI1/SPI2/ I2S2/I2S2ext	SPI3/I2Sext/ I2S3
Port B	PB0		TIM1_CH2N	TIM3_CH3	TIM8_CH2N			
	PB1		TIM1_CH3N	TIM3_CH4	TIM8_CH3N			
	PB2							
	PB3	JTDO/ TRACESWO	TIM2_CH2				SPI1_SCK	SPI3_SCK I2S3_CK
	PB4	NJTRST		TIM3_CH1			SPI1_MISO	SPI3_MISO
	PB5			TIM3_CH2		I2C1_SMBA	SPI1_MOSI	SPI3_MOSI I2S3_SD
	PB6			TIM4_CH1		I2C1_SCL		
	PB7			TIM4_CH2		I2C1_SDA		
	PB8			TIM4_CH3	TIM10_CH1	I2C1_SCL		
	PB9			TIM4_CH4	TIM11_CH1	I2C1_SDA	SPI2_NSS I2S2_WS	

注：并非每个引脚都有**16**种功能

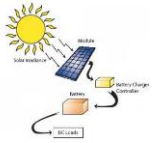


（5）STM32F407中断系统

STM32F407包含10个内核中断和82个可屏蔽中断通道。

中断向量在启动文件startup_stm32f40_41xxx.s中设置

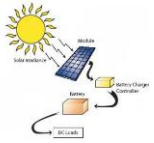
```
__Vectors       DCD     __initial_sp               ; Top of Stack
                DCD     Reset_Handler              ; Reset Handler
                DCD     NMI_Handler                ; NMI Handler
                DCD     HardFault_Handler          ; Hard Fault Handler
                DCD     MemManage_Handler          ; MPU Fault Handler
                DCD     BusFault_Handler           ; Bus Fault Handler
                DCD     UsageFault_Handler         ; Usage Fault Handler
                DCD     0                          ; Reserved
                DCD     0                          ; Reserved
                DCD     0                          ; Reserved
                DCD     0                          ; Reserved
                DCD     SVC_Handler                ; SVCall Handler
                DCD     DebugMon_Handler           ; Debug Monitor Handler
                DCD     0                          ; Reserved
                DCD     PendSV_Handler             ; PendSV Handler
                DCD     SysTick_Handler            ; SysTick Handler
```



中断和嵌套向量中断控制器NVIC

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

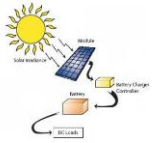


中断和嵌套向量中断控制器NVIC

中断使能寄存器，每个位控制一个中断，置1表示允许该中断。

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];       /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];       /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];       /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];       /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];       /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;          /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

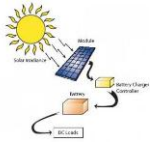


中断和嵌套向量中断控制器NVIC

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

中间预留24个字单元

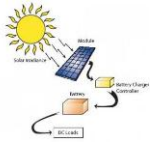


中断和嵌套向量中断控制器NVIC

中断除能寄存器，每个位控制一个中断，置1表示禁止该中断。

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

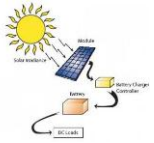


中断和嵌套向量中断控制器NVIC

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

中断挂起寄存器，置1表示挂起该中断

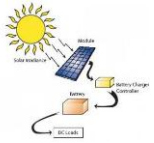


中断和嵌套向量中断控制器NVIC

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

中断解挂寄存器，置1表示解除该中断挂起标志

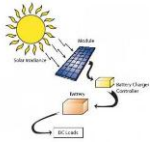


中断和嵌套向量中断控制器NVIC

NVIC寄存器定义

```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

中断激活寄存器，置1表示该中断正在执行



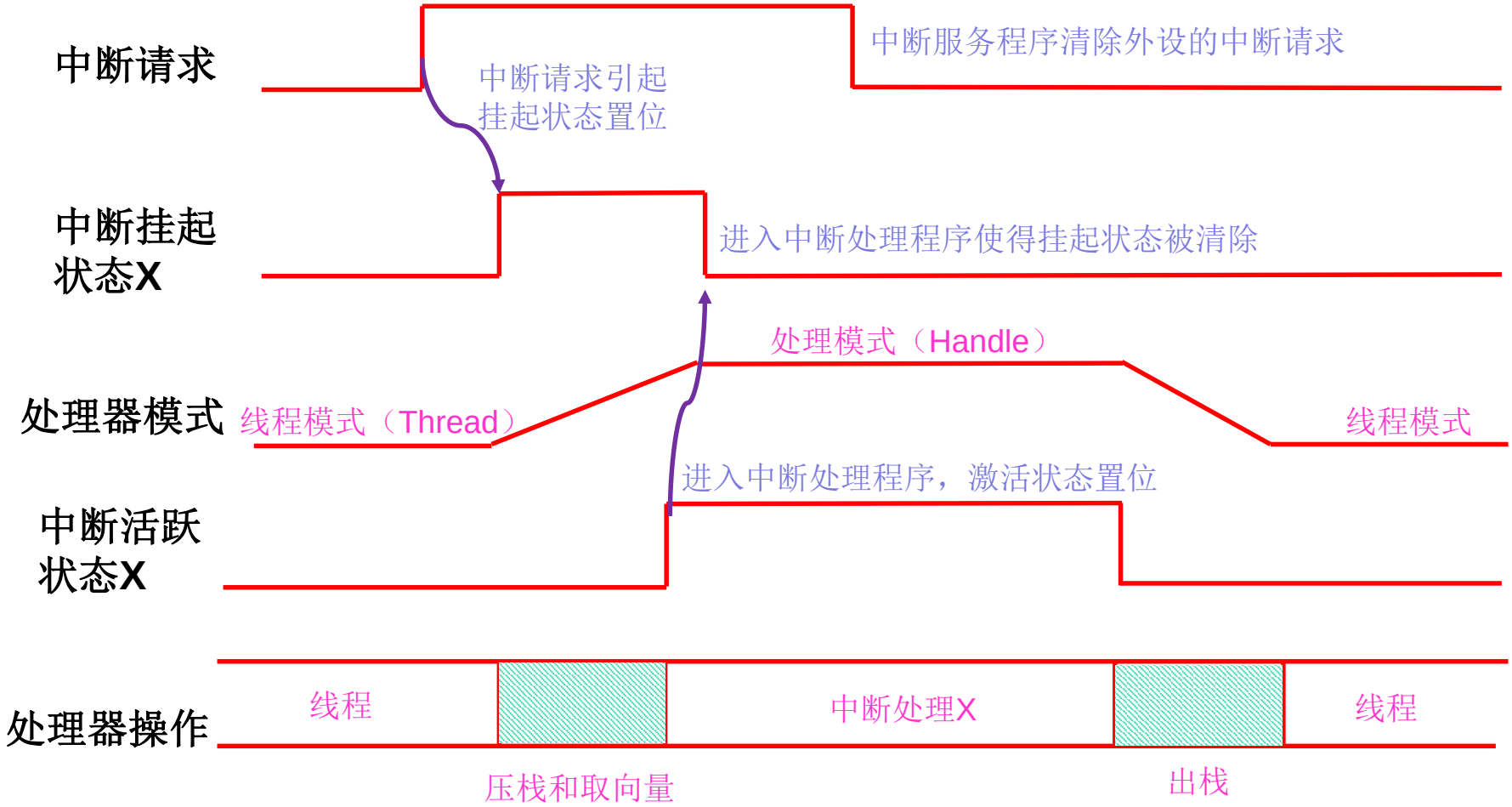
中断和嵌套向量中断控制器NVIC

NVIC寄存器定义

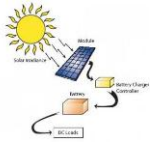
```
typedef struct
{
  __IO uint32_t ISER[8];        /*!< Offset: 0x000 (R/W)  Interrupt Set Enable Register     */
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];        /*!< Offset: 0x080 (R/W)  Interrupt Clear Enable Register  */
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];        /*!< Offset: 0x100 (R/W)  Interrupt Set Pending Register    */
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];        /*!< Offset: 0x180 (R/W)  Interrupt Clear Pending Register*/
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];        /*!< Offset: 0x200 (R/W)  Interrupt Active bit Register       */
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];    /*!< Offset: 0x300 (R/W)  Interrupt Priority Register (8Bit wide) */
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;           /*!< Offset: 0xE00 ( /W)  Software Trigger Interrupt Register  */
}  NVIC_Type;
```

中断优先级控制，每个中断占1个字节，但是仅高4位有效

中断的挂起和激活



注：Cortex-M 两种操作模式：线程模式和处理模式
在处理模式下，处理器总是具有特权访问等级；
在线程模式下，处理器可处于特权访问等级，也可处于非特权访问等级，由CONTROL寄存器设置。



NVIC的中断优先级

优先级包括抢占优先级和响应优先级：

- 共占4bit，哪几个bit表示抢占优先级或响应优先级可设置；
- 编号越小，优先级越高；
- 抢占优先级高的中断可以打断优先级低的中断；
- 抢占优先级相同时，两个中断同时到达，则先处理响应优先级高的中断。

例：

中断3的抢占优先级为2，响应优先级为1；

中断6的抢占优先级为3，响应优先级为0；

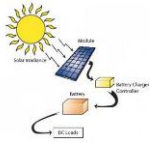
中断7的抢占优先级为2，响应优先级为0；

则

中断7和中断3可打断中断6；

中断7和中断3不可互相打断；

中断7和中断3同时触发时响应中断7；



NVIC的中断优先级

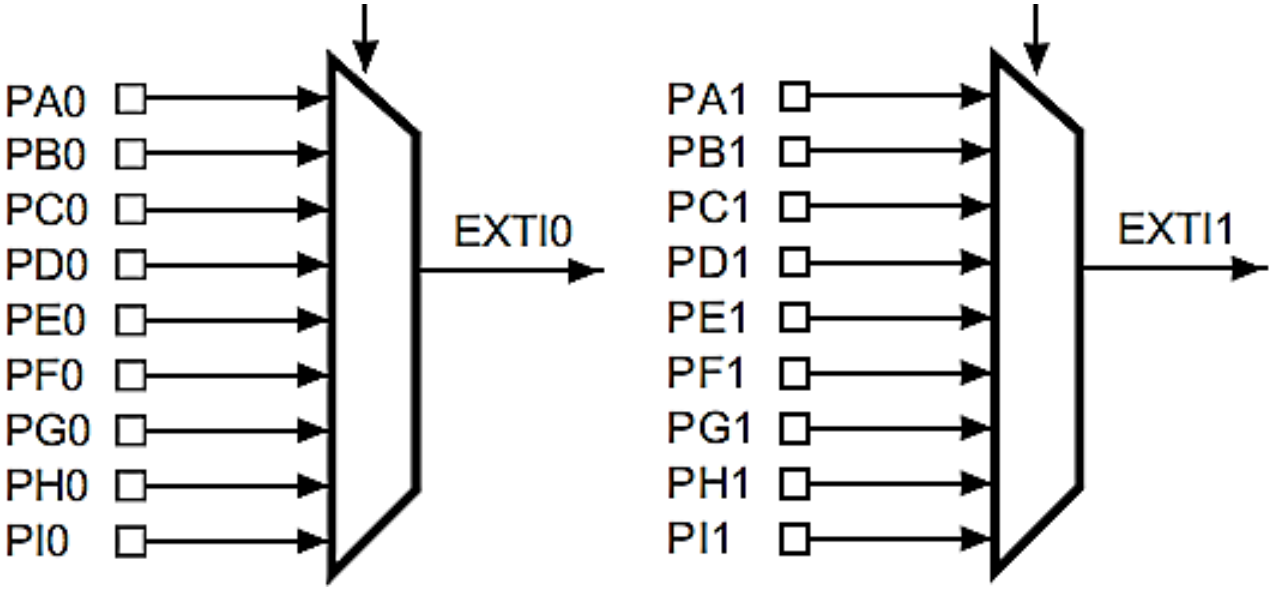
优先级寄存器NVIC->IP的有效位数为4位Bit[7:4]，包括抢占优先级和响应优先级的位数。
具体分配情况由 SCB->AIRCR 寄存器的 bit10~8 定义

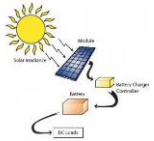
	AIRCR[10:8]	Bit[7:4]分配	分配结果
0	111	0：4	0位抢占优先级，4位响应优先级
1	110	1：3	1位抢占优先级，3位响应优先级
2	101	2：2	2位抢占优先级，2位响应优先级
3	100	3：1	3位抢占优先级，1位响应优先级
4	011	4：0	4位抢占优先级，0位响应优先级

外部中断EXTI

特点：

- **23个中断源，其中IO口输入的中断线有16路；**
- **每个IO口都可以作为中断输入；**
- **GPIOA.0，GPIOB.0，……，GPIOI.0复用一个中断输入；**





外部中断EXTI编程

采用库函数编程：GPIOD15触发中断

外中断源初始化

对GPIOD15初始化

```
SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOD, EXTI_PinSource15);
EXTI_InitStructure.EXTI_Line =  EXTI_Line15;
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
EXTI_Init(&EXTI_InitStructure);
```

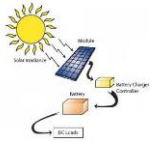
设置为上升沿触发

中断初始化

设置抢占优先级

设置响应优先级

```
NVIC_InitStructure.NVIC_IRQChannel = EXTI15_10_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x03;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0x03;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);
```



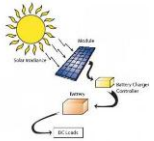
NVIC中断程序

启动文件**startup_stm32f40_41xxx.s**中：

```
DCD    EXTI15_10_IRQHandler          ; External Line[15:10]s
```

中断程序：

```
void EXTI15_10_IRQHandler(void)
{
        if(EXTI_GetITStatus(EXTI_Line15) != RESET)
        {
        EXTI_ClearITPendingBit(EXTI_Line15); //清外中断标志
         ……
        }
}
```



（6）定时器

- 14个定时器

  定时器1和8：高级控制定时器，16位自动重装载；

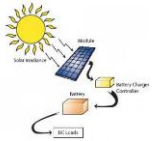
  定时器2和5：通用定时器，32位自动重装载；

  定时器3和4：通用定时器， 16位自动重装载；

  定时器6和7：基本定时器， 16位自动重装载；

  定时器9-14：通用定时器， 16位自动重装载；

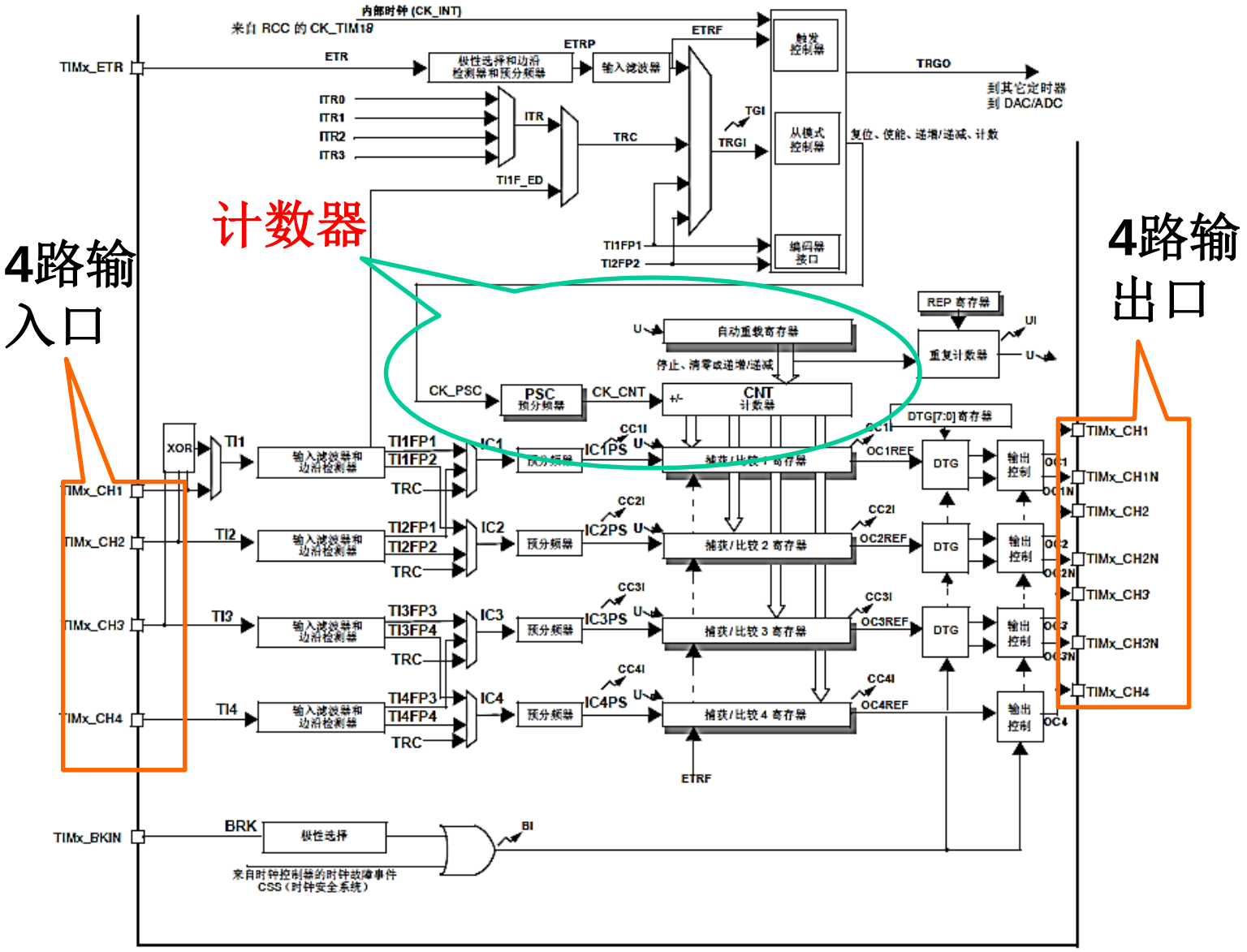
- 定时器相互完全独立，不共享任何资源；
- 功能强大，可用于定时、计数、PWM控制等；



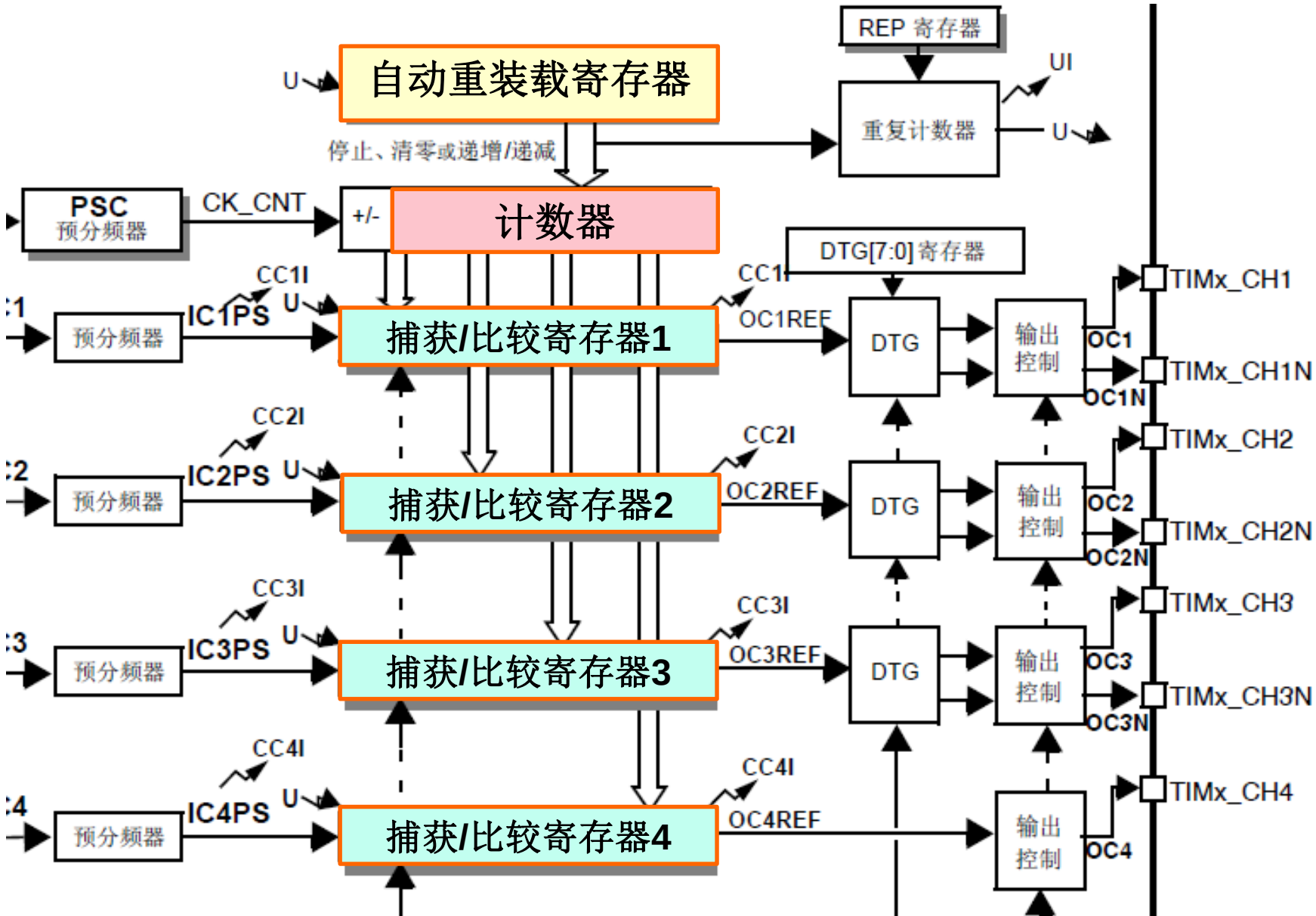
TIM1和TIM8特性

- 16位递增、递减、递增/递减自动重装载计数器；
- 16位可编程预分频器，对计数器频率进行分频；
- 4个输入或输出通道，可用于
  - —输入捕获
  - —输出比较
  - —PWM生成
- 带死区的PWM互补输出
- 可实现多个定时器之间的同步互连

TIM1和TIM8框图



TIM1和TIM8框图



三种计数方式

计数器值

自动重装载值
TIMx_ARR

递增计数

TIMx_CNT

TIMx_ARR

0

周期数：ARR+1

递减计数

TIMx_CNT

TIMx_ARR

0

周期数：ARR+1

中心对齐模式

TIMx_CNT

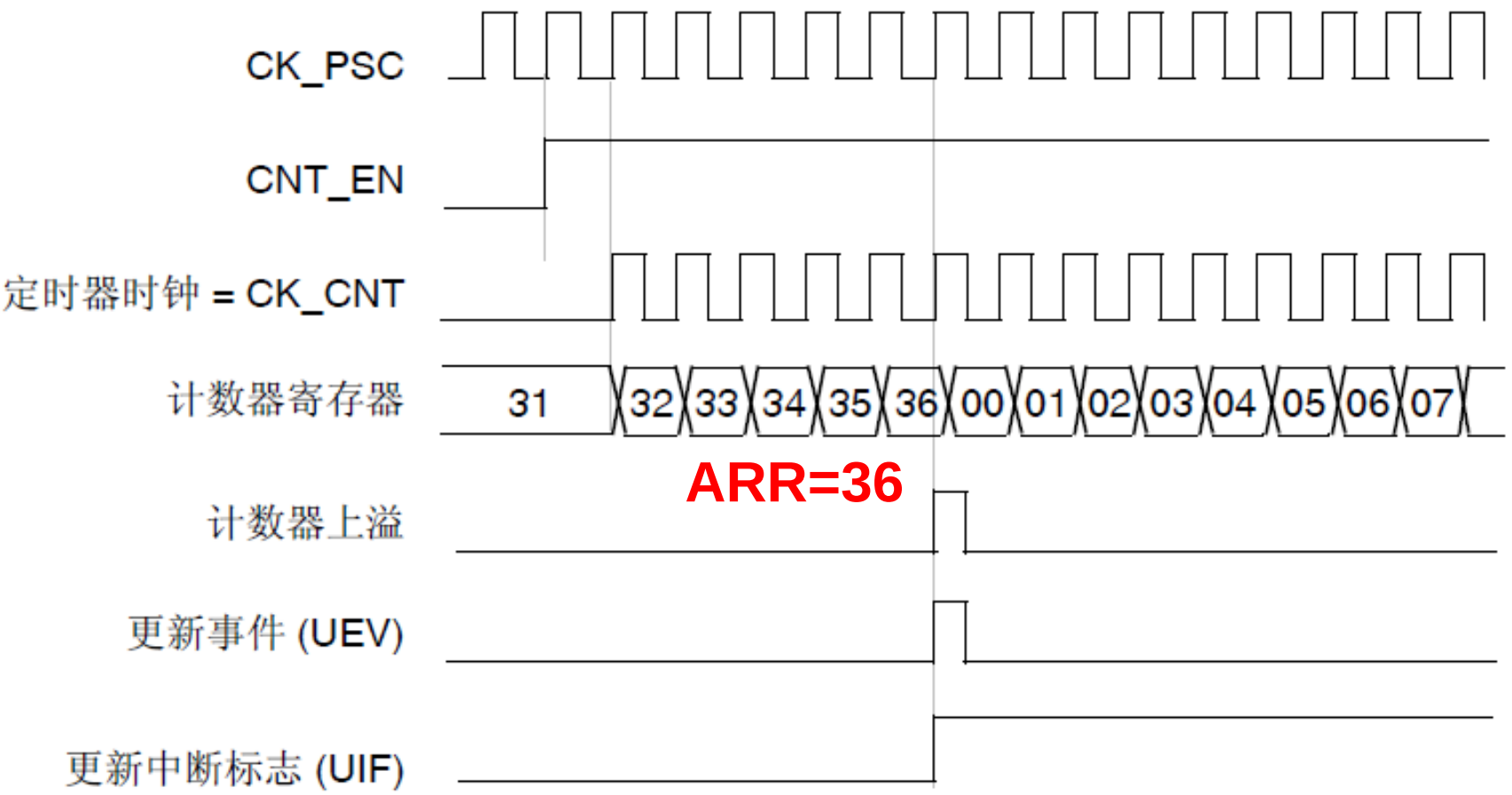
TIMx_ARR

0

周期数：2*ARR

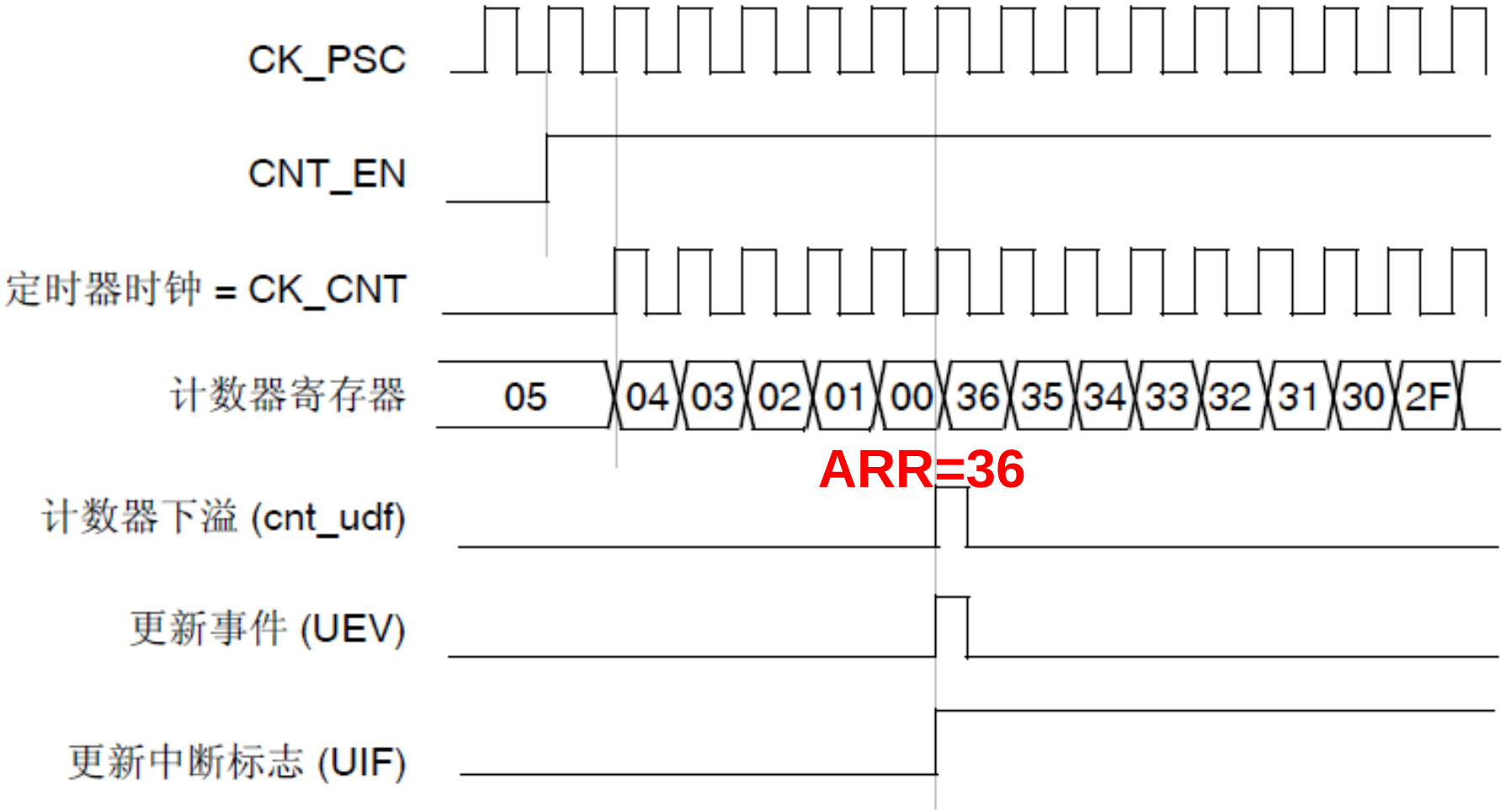
计数器时序

1、递增方式时序（1分频内部时钟）



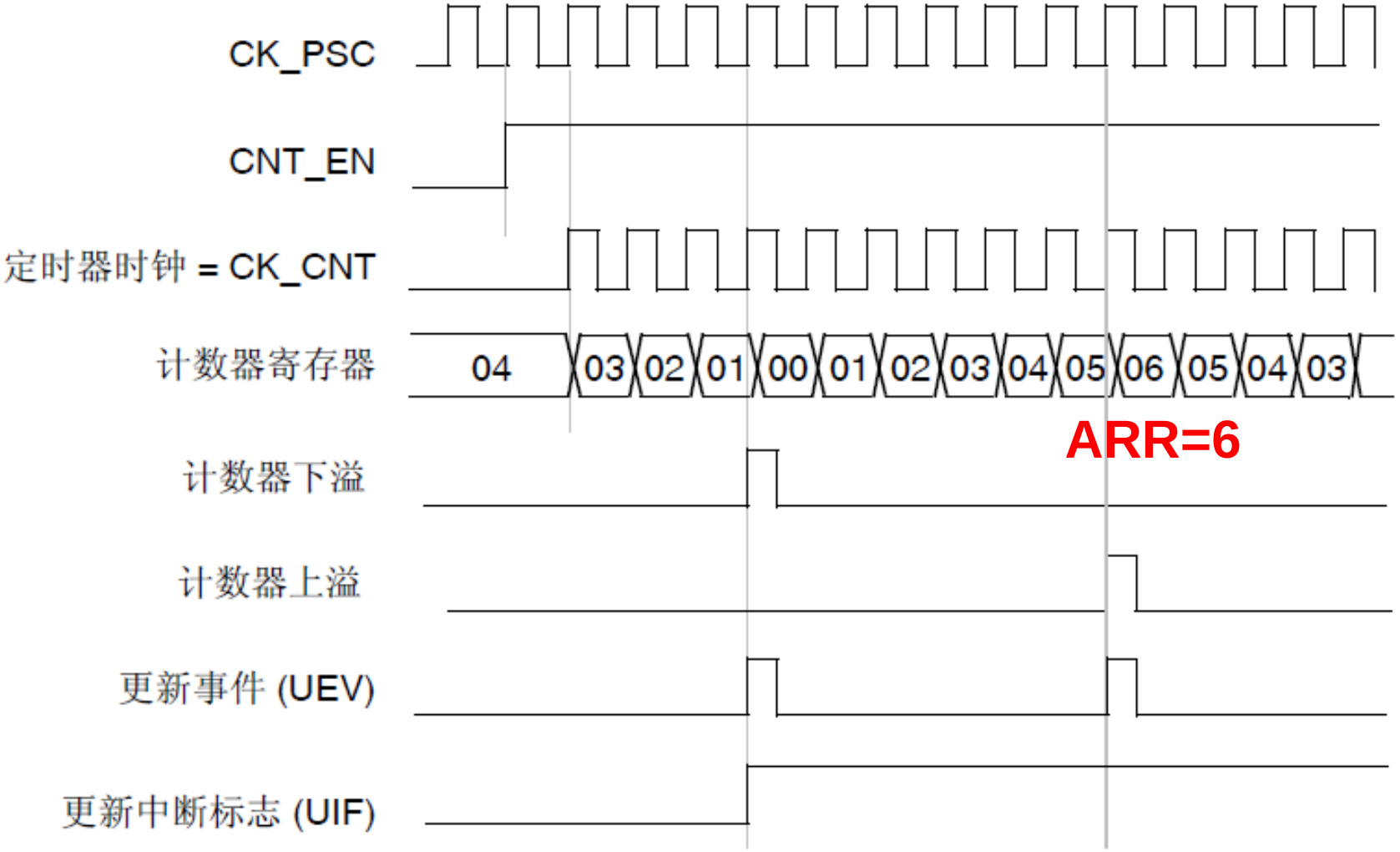
计数器时序

2、递减方式时序（1分频内部时钟）

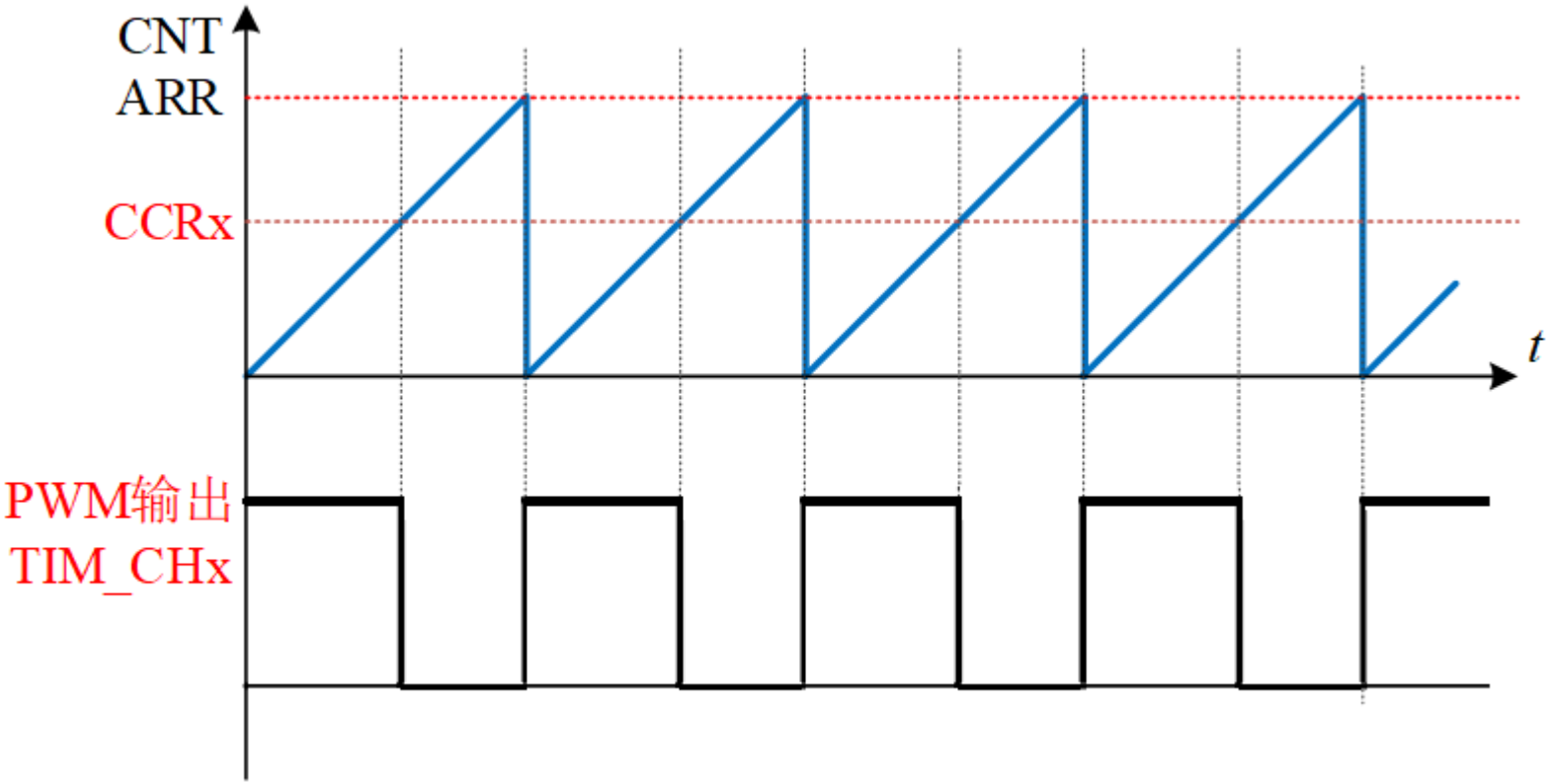


计数器时序

3、中心对齐方式时序（1分频内部时钟）



PWM工作方式



PWM设计原理：

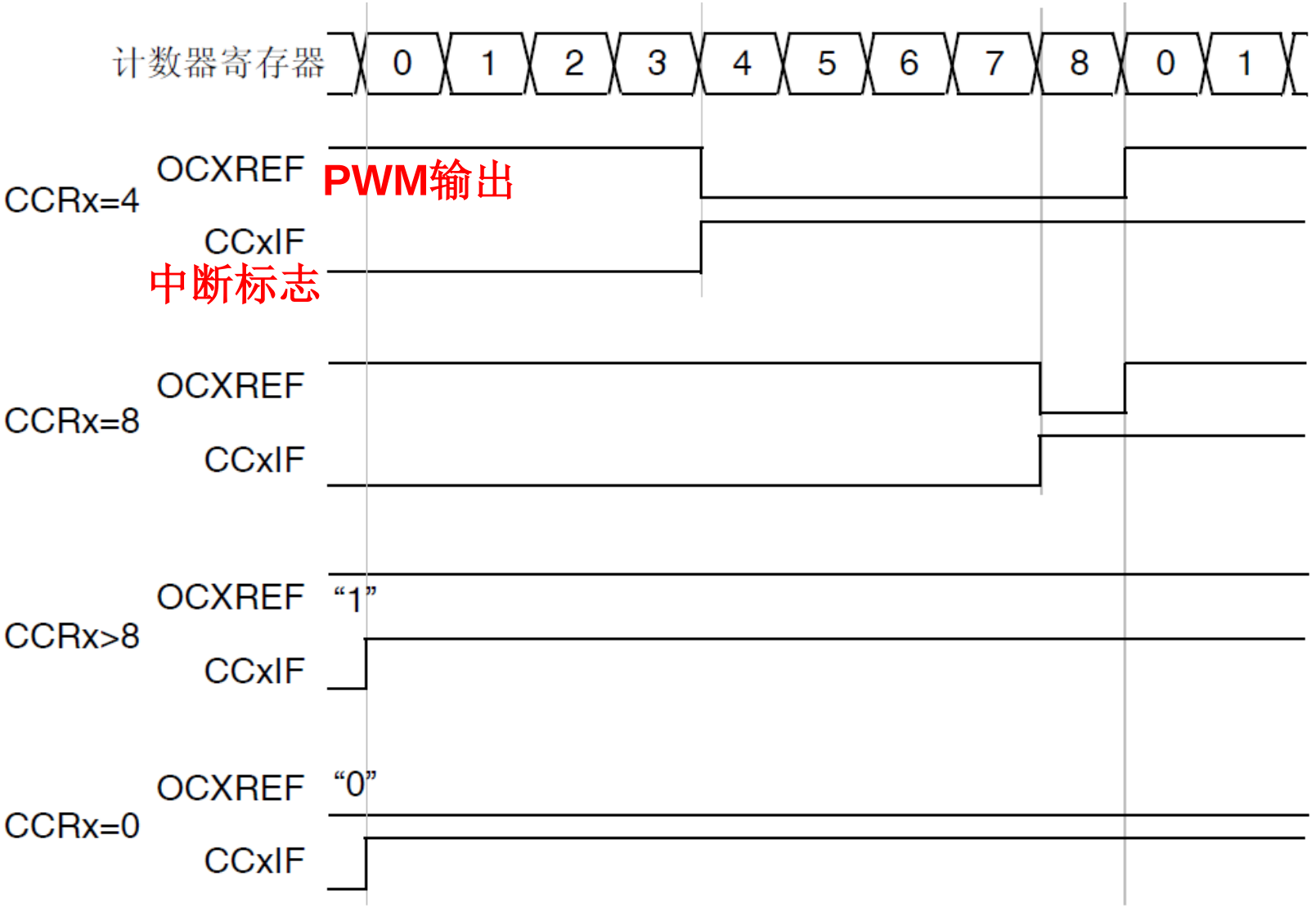
1、设置计数器向上/向下计数等工作方式；

2、设置计数周期（TIMx_ARR寄存器）；

3、设置计数比较值（TIM_CCRx, x=1,2,3,4）

PWM工作方式

递增方式下的PWM波形（ARR=8）



PWM程序设计

例：

1、采用定时器TIM1在CH1引脚输出1kHz的PWM波形，芯片采用STM32F407VET6，LQFP100封装，主频168MHz。

（1）选择GPIO口

Table 8. Alternate function mapping

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF014	AF15
		SYS	TIM1/2	TIM3/4/5	TIM8/9/10/11	I2C1/2/3	SPI1/SPI2/ I2S2/I2S2ext	SPI3/I2S2ext/ I2S3	USART1/2/3/ I2S3ext	UART4/5/ USART6	CAN1/CAN2/ TIM12/13/14	OTG_FS/ OTG_HS	ETH	FSMC/SDIO/ OTG_FS	DCMI		
Port A	PA0		TIM2_CH1 TIM2_ETR	TIM 5_CH1	TIM8_ETR				USART2_CTS	UART4_TX			ETH_MII_CRS				EVENTOUT
	PA1		TIM2_CH2	TIM5_CH2					USART2_RTS	UART4_RX			ETH_MII_RX_CLK ETH_RMII__REF_ CLK				EVENTOUT
	PA2		TIM2_CH3	TIM5_CH3	TIM9_CH1				USART2_TX				ETH_MDIO				EVENTOUT
	PA3		TIM2_CH4	TIM5_CH4	TIM9_CH2				USART2_RX			OTG_HS_ULPI_D0	ETH_MII_COL				EVENTOUT
	PA4						SPI1_NSS	SPI3_NSS I2S3_WS	USART2_CK					OTG_HS_SOF	DCMI_HSYNC		EVENTOUT
	PA5		TIM2_CH1 TIM2_ETR		TIM8_CH1N		SPI1_SCK					OTG_HS_ULPI_CK					EVENTOUT
	PA6		TIM1_BKIN	TIM3_CH1	TIM8_BKIN		SPI1_MISO				TIM13_CH1				DCMI_PIXCK		EVENTOUT
	PA7		TIM1_CH1N	TIM8_CH2	TIM8_CH1N		SPI1_MOSI				TIM14_CH1		ETH_MII_RX_DV ETH_RMII_CRS_DV				EVENTOUT
	PA8	MCO1	TIM1_CH1			I2C3_SCL			USART1_CK			OTG_FS_SOF					EVENTOUT
	PA9		TIM1_CH2			I2C3_SMBA			USART1_TX						DCMI_D0		EVENTOUT
	PA10		TIM1_CH3						USART1_RX			OTG_FS_ID			DCMI_D1		EVENTOUT
	PA11		TIM1_CH4						USART1_CTS		CAN1_RX	OTG_FS_DM					EVENTOUT
	PA12		TIM1_ETR						USART1_RTS		CAN1_TX	OTG_FS_DP					EVENTOUT
	PA13	JTMS-SWDIO															EVENTOUT
	PA14	JTCK-SWCLK															EVENTOUT
	PA15	JTDI	TIM 2_CH1 TIM 2_ETR				SPI1_NSS	SPI3_NSS/ I2S3_WS									EVENTOUT

PWM程序设计

例：

1、采用定时器TIM1在CH1引脚输出1kHz的PWM波形，芯片采用STM32F407VET6，LQFP100封装，主频168MH。

（1）选择GPIO口，PA8的AF1

Port A	PA5		TIM2_ETR		TIM8_CH1N
	PA6		TIM1_BKIN	TIM3_CH1	TIM8_BKIN
	PA7		TIM1_CH1N	TIM3_CH2	TIM8_CH1N
	PA8	MCO1	TIM1_CH1		
	PA9		TIM1_CH2		
	PA10		TIM1_CH3		
	PA11		TIM1_CH4		

PWM程序设计

初始化：

（1）GPIO初始化

```
GPIO_InitTypeDef    GPIO_InitStructure;
RCC_AHB1PeriphClockCmd( RCC_AHB1Periph_GPIOA, ENABLE);
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP ;
GPIO_Init(GPIOA, &GPIO_InitStructure);
GPIO_PinAFConfig(GPIOA, GPIO_PinSource8,GPIO_AF_TIM1);
```

打开GPIOA的时钟

选PIN8

选特殊功能

特殊功能栏选TIM1

PWM程序设计

（2）TIM1初始化

```
TIM_OCInitTypeDef  TIM_OCInitStructure;
TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1 , ENABLE);
TIM_TimeBaseStructure.TIM_Prescaler= 168-1;
TIM_TimeBaseStructure.TIM_Period = 1000-1;
TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;
TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);
```

打开TIM1的时钟

168预分频后输出1MHz

周期ARR设置1ms

```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
TIM_OCInitStructure.TIM_Pulse =500;
TIM_OC1Init(TIM1, &TIM_OCInitStructure);
```

PWM程序设计

（2）TIM1初始化

```
TIM_OCInitTypeDef  TIM_OCInitStructure;
TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1 , ENABLE);
TIM_TimeBaseStructure.TIM_Prescaler= 168-1;
TIM_TimeBaseStructure.TIM_Period = 1000-1;
TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;
TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);
```

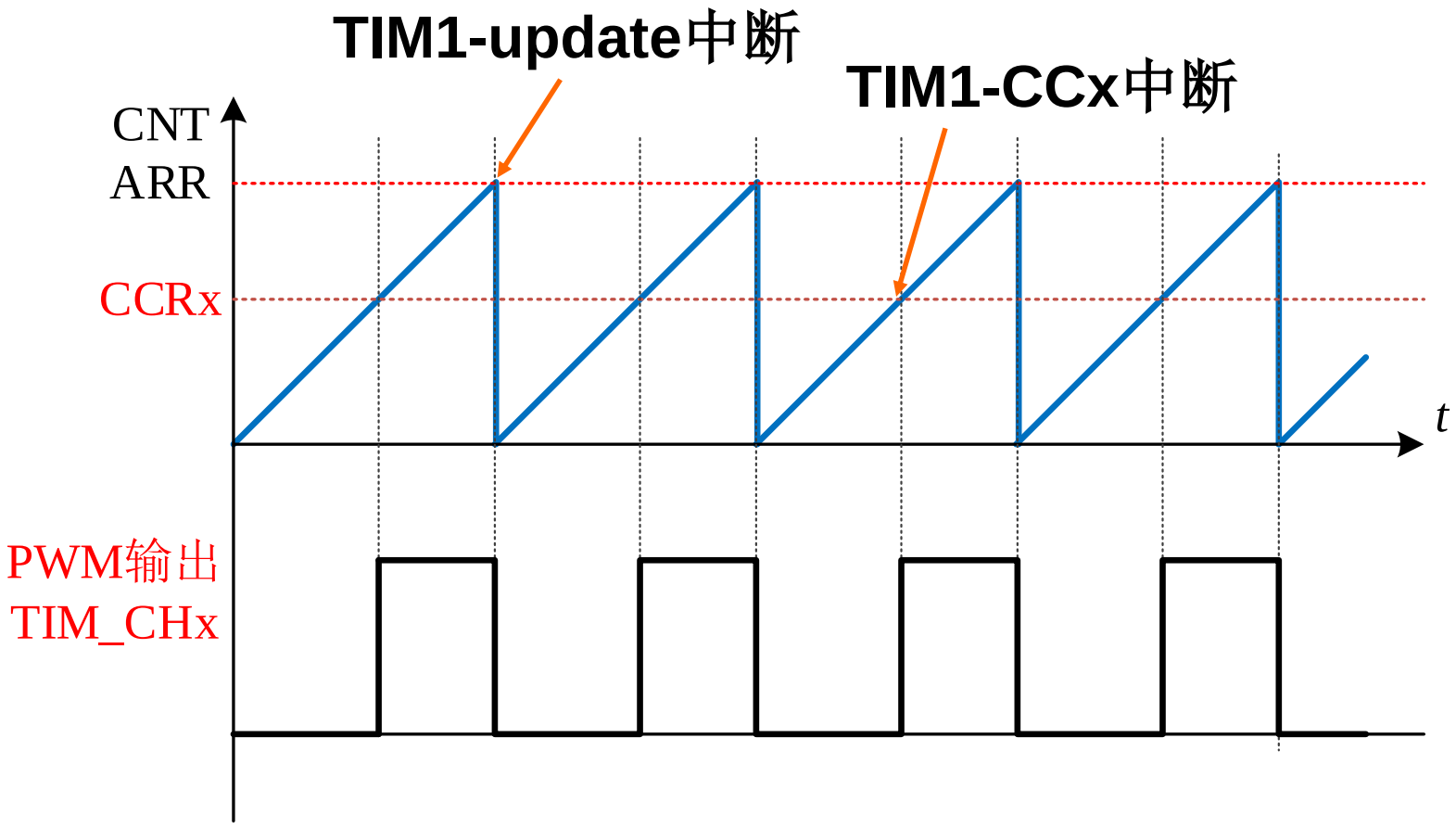
PWM模式设置

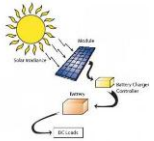
```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
TIM_OCInitStructure.TIM_Pulse =500;
TIM_OC1Init(TIM1, &TIM_OCInitStructure);
TIM_CtrlPWMOutputs(TIM1, ENABLE);
```

PWM开通时间
CCR1寄存器设置

PWM程序设计

（2）中断源





PWM程序设计

（2）TIM1中断服务程序

```
void TIM1_UP_TIM10_IRQHandler(void)
{
        if(TIM_GetITStatus(TIM1,TIM_IT_Update) != RESET)
        {
                TIM_ClearITPendingBit(TIM1,TIM_FLAG_Update);
                .......

        }
}
```

判断是否TIM1中断

清除中断标志



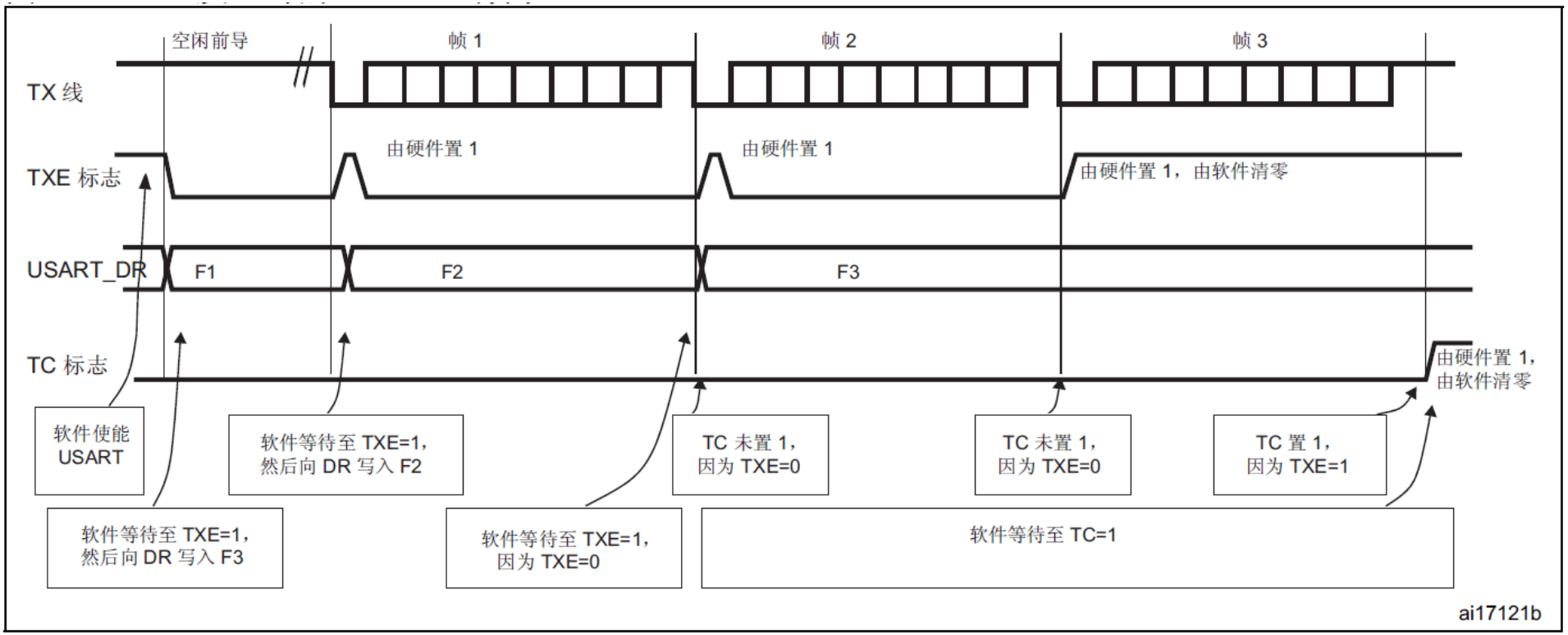
（7）通用同步异步收发器 (USART)

- 全双工异步通信
- NRZ 标准格式（标记/空格）
- 可配置为 16 倍过采样或 8 倍过采样
- 通用可编程收发波特率
- 数据字长度可编程（8 位或 9 位）
- 停止位可配置 - 支持 1 或 2 个停止位
- 十个具有标志位的中断源：

其中常用的中断：

- 发送数据寄存器为空（TXE）
- 发送完成 （TC）
- 接收到数据（RXNE）
- 接收到线路空闲（IDLE）

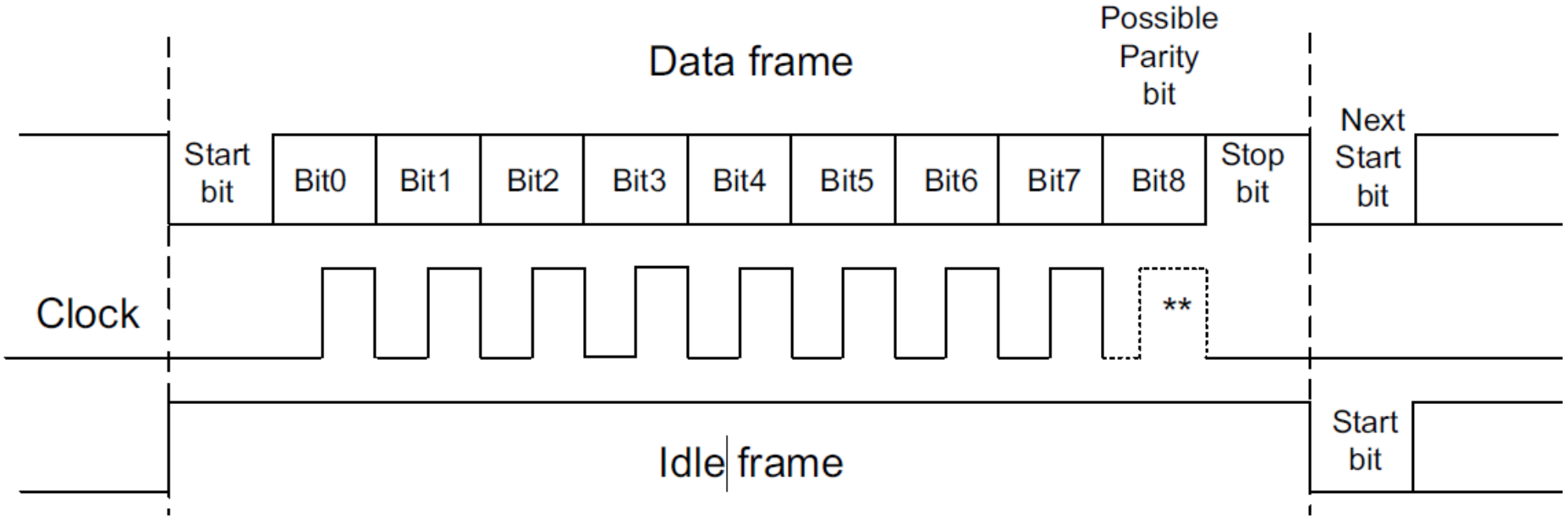
（7）通用同步异步收发器 (USART)



- 发送数据的TXE和TC标志
- TXE是单个字节发送完毕中断
- TC是批量数据发送完成中断

（7）通用同步异步收发器 (USART)

IDLE中断原理

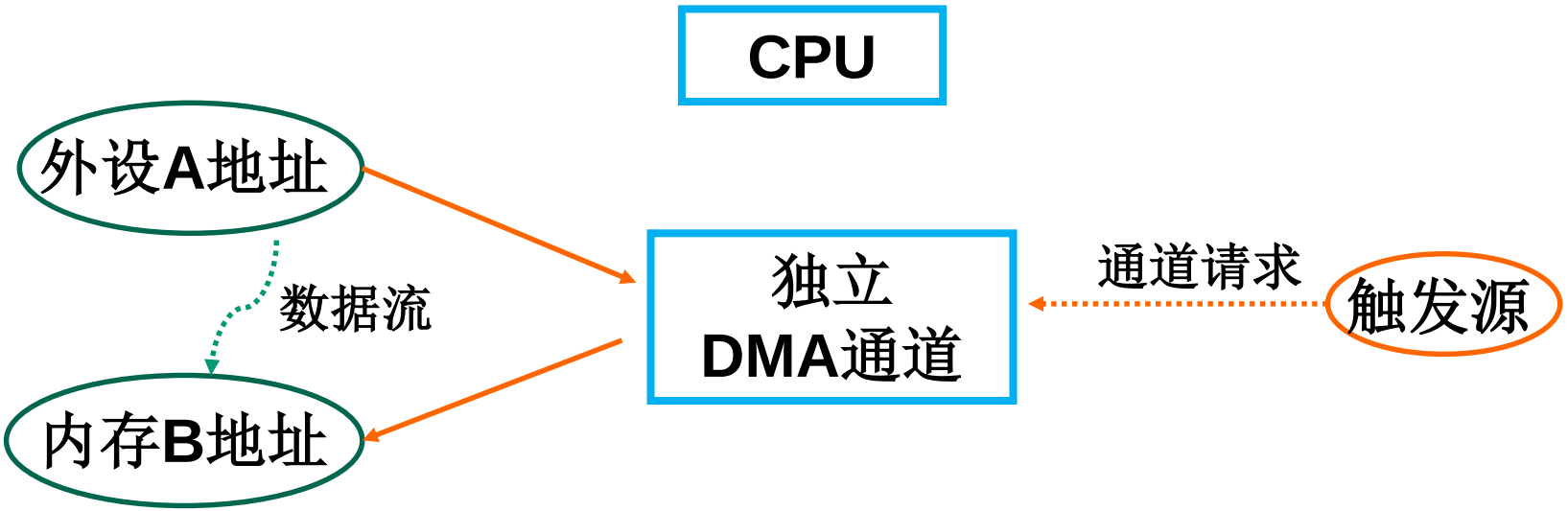


- IDLE中断是在监测到数据接收后（即串口的RXNE位被置位）开始检测，当总线上在一个字节对应的周期内未再有新的数据接收时，触发空闲中断IDLE位被硬件置1.
- IDLE中断用于数据的批量接收。

（8）直接内存访问DMA

什么是DMA（Direct Memory Access）

- 定义和本质：是一种硬件机制，允许外设直接与内存交换数据，而无需经过 CPU 的干预。它在总线上充当“第二主控器”。
- 核心使命：将 CPU 从繁琐、机械的“数据搬运”任务中解放出来，使其能够专注于复杂的算法计算和逻辑控制，实现真正的并行处理。



（8）直接内存访问DMA

核心原理：总线矩阵与主控权

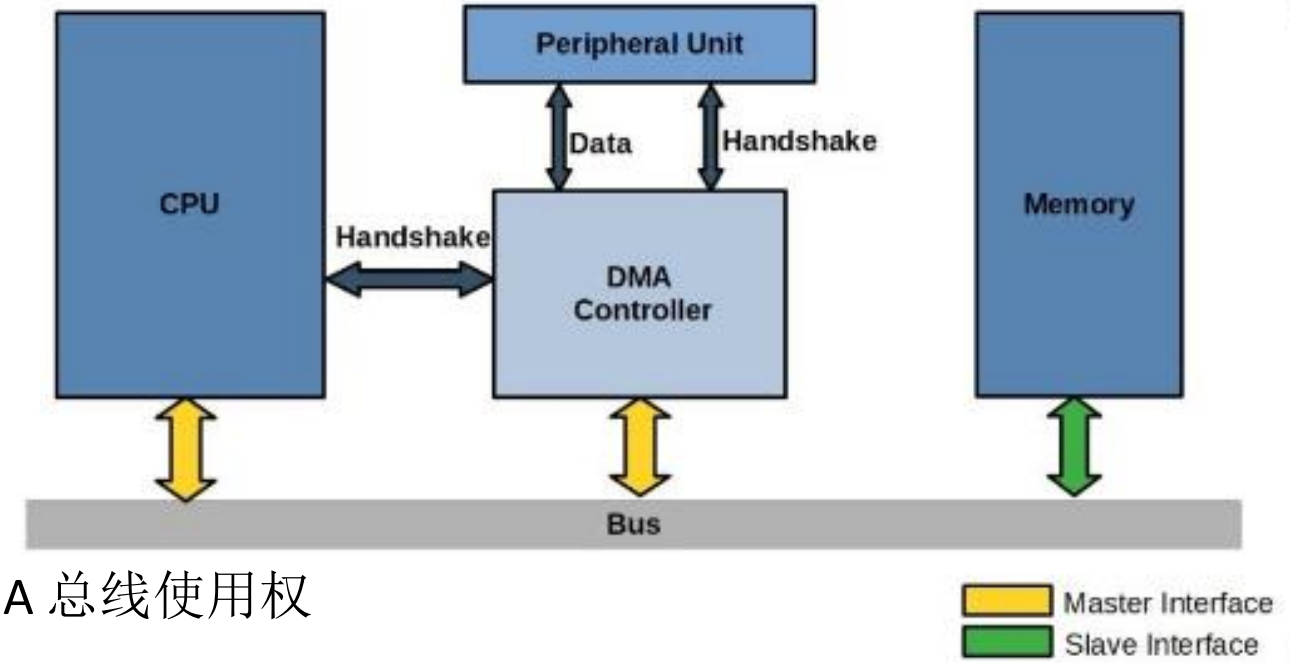
DMA 控制器通过总线矩阵（Bus Matrix）与 CPU 共享系统资源

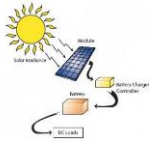
DMA 工作过程

- **请求和握手：**
外设触发 DMA 请求

- **仲裁授权：**
总线仲裁器授予 DMA 总线使用权

- **自主传输：**
DMA 独立生成地址并驱动数据





（8）直接内存访问DMA

DMA的优点

CPU 卸载

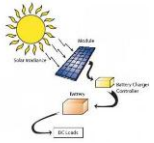
消除中断处理开销，避免每次数据传输都触发一次中断。释放 CPU 计算资源，提升系统整体吞吐量。

提高实时性

通过硬件级的快速响应，确保高频外设（如 ADC、SPI）的数据流不丢失，降低系统延迟抖动。

降低功耗

数据传输期间 CPU 可进入睡眠模式（Sleep Mode），仅由低功耗 DMA 完成搬运，大幅延长电池寿命。



（8）直接内存访问DMA

性能对比：CPU 资源占用率

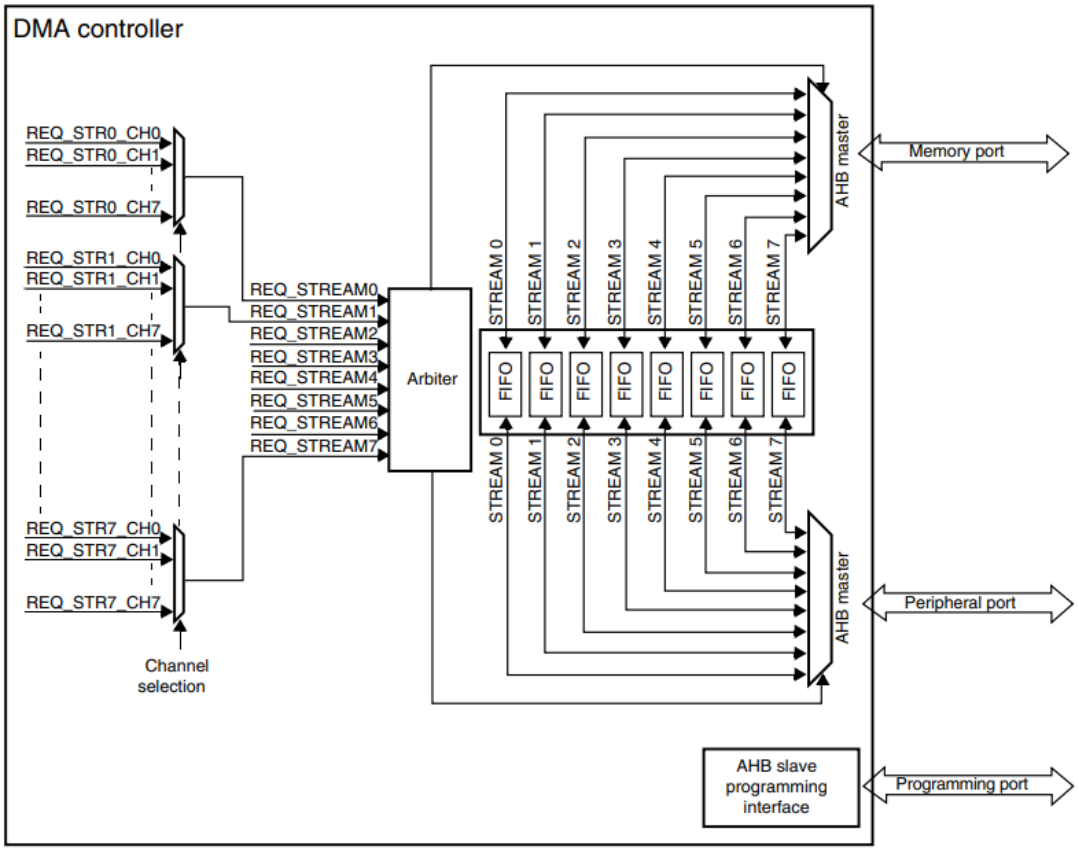


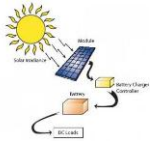
在高数据吞吐量场景下，**DMA 可将 CPU 从繁琐的搬运中完全释放**

（8）直接内存访问DMA

DMA控制要素

1. 触发源：
外设或软件触发信号
2. 源地址：
读取数据的内存或外设地址。
3. 目标地址：
写入内存或外设的目标地址
4. DMA通道：
设置DMA内部 FIFO 通道





（8）直接内存访问DMA

STM32F4具有2个DMA控制器（DMA1和DMA2），共16个数据流（每个控制器8路），每个数据流有8个通道，大部分外设均可通过DMA控制。

DMA的四种传输方式：

外设到内存

内存到外设

内存到内存

外设到外设

需要设置的参数：

触发源及其通道

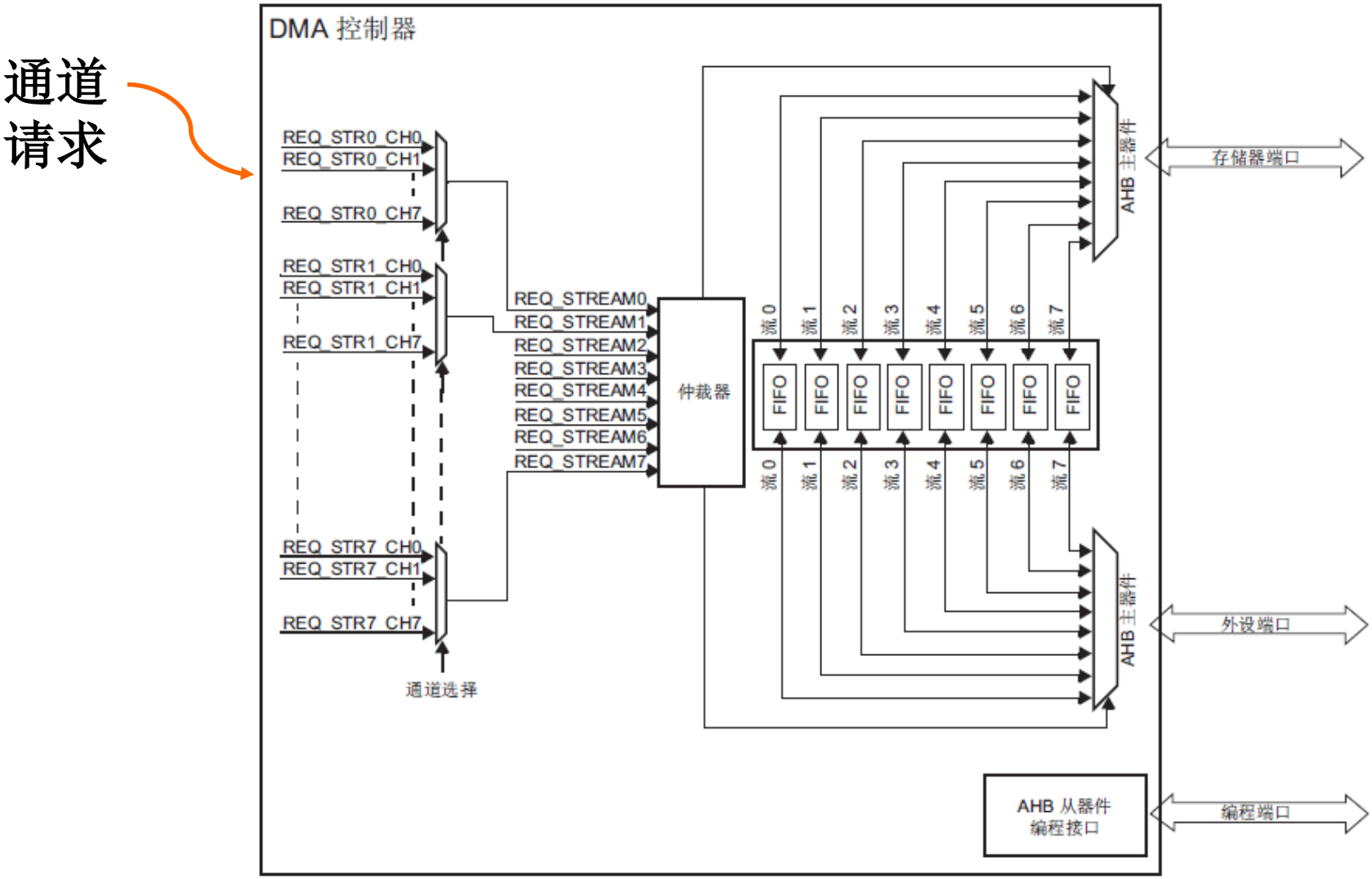
源地址

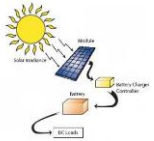
目标地址

传输数据类型、长度

（8）直接内存访问DMA

STM32F4的DMA结构框图





（8）直接内存访问DMA

DMA应用1：

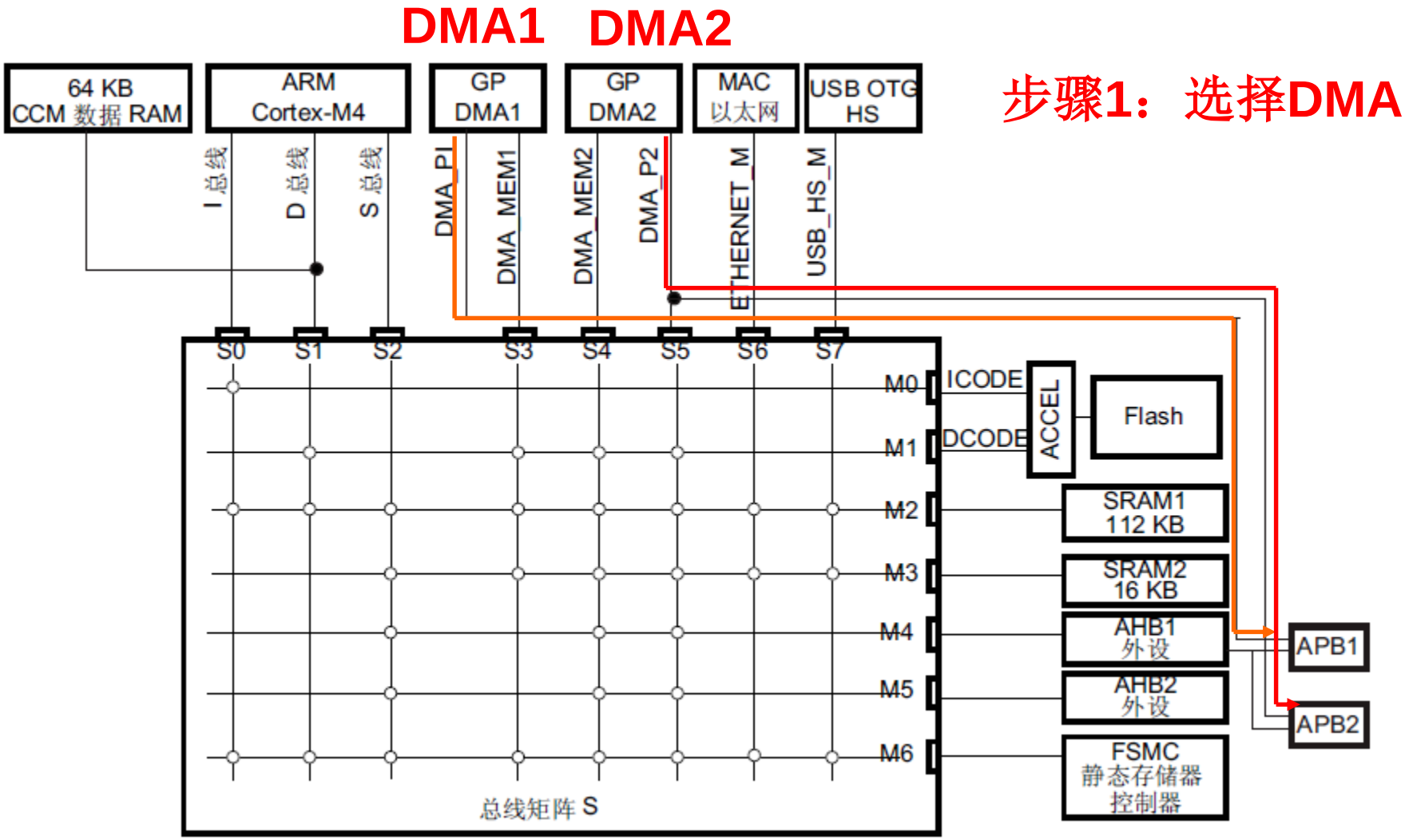
对输入波形连续高速AD采样，完成全部数据采集后批量处理；

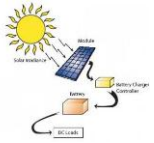
DMA应用2：

设计任意波形发生器，将表格中的数据直接送给DA转换，不经CPU程序处理。

（8）直接内存访问DMA

DMA设计示例：将UART1接收的数据放到SRAM





（8）直接内存访问DMA

表 2. STM32F4xx 寄存器边界地址

边界地址	外设	总线	寄存器映射
0xA000 0000 - 0xA000 0FFF	FSMC 控制寄存器	AHB3	*第 1241 页的第 32.6.9 节：FSMC 寄存器映射*
0x5006 0800 - 0x5006 0BFF	RNG	AHB2	*第 549 页的第 21.4.4 节：RNG 寄存器映射*
0x5006 0400 - 0x5006 07FF	HASH		*第 569 页的第 22.4.9 节：散列寄存器映射*
0x5006 0000 - 0x5006 03FF	CRYP		*第 543 页的第 20.6.13 节：CRYP 寄存器映射*
0x5005 0000 - 0x5005 03FF	DCMI		*第 327 页的第 13.8.12 节：DCMI 寄存器映射*
0x5000 0000 - 0x5003 FFFF	USB OTG FS		*第 1006 页的第 30.16.6 节：OTG_FS 寄存器映射*
0x4002 6400 - 0x4002 67FF	DMA2		*第 229 页的第 9.5.11 节：DMA 寄存器映射*
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		
0x4002 3C00 - 0x4002 3FFF	Flash 接口寄存器		*第 3.8 节：Flash 接口寄存器*
0x4002 3800 - 0x4002 3BFF	RCC		*第 171 页的第 6.3.32 节：RCC 寄存器映射*
0x4002 3000 - 0x4002 33FF	CRC	AHB1	*第 85 页的第 4.4.4 节：CRC 寄存器映射*
0x4002 2000 - 0x4002 23FF	GPIOI		
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4001 1400 - 0x4001 17FF	USART6	APB2	*第 720 页的第 26.6.8 节：USART 寄存器映射*
0x4001 1000 - 0x4001 13FF	USART1		
0x4001 0400 - 0x4001 07FF	TIM8		*第 390 页的第 14.4.21 节：TIM1 和 TIM8 寄存器映射*
0x4001 0000 - 0x4001 03FF	TIM1		

对应总线

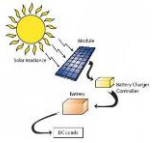
（8）直接内存访问DMA

表 36. DMA2 请求映射

外设请求	数据流 0	数据流 1	数据流 2	数据流 3	数据流 4	数据流 5	数据流 6	数据流 7
通道 0	ADC1		TIM8_CH1 TIM8_CH2 TIM8_CH3		ADC1		TIM1_CH1 TIM1_CH2 TIM1_CH3	
通道 1		DCMI	ADC2	ADC2		SPI6_TX(1)	SPI6_RX(1)	DCMI
通道 2	ADC3	ADC3		SPI5_RX(1)	SPI5_TX(1)	CRYP_OUT	CRYP_IN	HASH_IN
通道 3	SPI1_RX		SPI1_RX	SPI1_TX		SPI1_TX		
通道 4	SPI4_RX(1)	SPI4_TX(1)	USART1_RX	SDIO		USART1_RX	SDIO	USART1_TX
通道 5		USART6_RX	USART6_RX	SPI4_RX(1)	SPI4_TX(1)		USART6_TX	USART6_TX
通道 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
通道 7		TIM8_UP	TIM8_CH1	TIM8_CH2	TIM8_CH3	SPI5_RX(1)	SPI5_TX(1)	TIM8_CH4 TIM8_TRIG TIM8_COM

步骤2：选择数据流和通道

步骤3：编程



（8）直接内存访问DMA

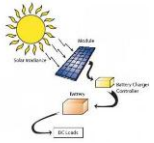
初始化设置：

1、串口初始化；

2、DMA初始化

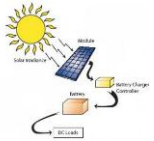
```
DMA_InitTypeDef DMA_InitStructure;
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2, ENABLE);

//DMA属性设置
DMA_InitStructure.DMA_Channel          = DMA_Channel_4;
DMA_InitStructure.DMA_PeripheralBaseAddr
                                       = (uint32_t)(&USART1->DR);
DMA_InitStructure.DMA_Memory0BaseAddr
                                       = (uint32_t)(&RS232_buff);
DMA_InitStructure.DMA_DIR
                                     = DMA_DIR_PeripheralToMemory;
DMA_InitStructure.DMA_BufferSize
                                      = REC_BUFF_SIZE;
```



（8）直接内存访问DMA

```
DMA_InitStructure.DMA_PeripheralInc     = DMA_PeripheralInc_Disable;
DMA_InitStructure.DMA_MemoryInc         = DMA_MemoryInc_Enable;
DMA_InitStructure.DMA_PeripheralDataSize
                                        = DMA_PeripheralDataSize_Byte;
DMA_InitStructure.DMA_MemoryDataSize
                                        = DMA_MemoryDataSize_Byte;
DMA_InitStructure.DMA_Mode              = DMA_Mode_Circular;
DMA_InitStructure.DMA_Priority          = DMA_Priority_VeryHigh;
DMA_InitStructure.DMA_FIFOMode      = DMA_FIFOMode_Disable;
DMA_InitStructure.DMA_FIFOThreshold
                                    = DMA_FIFOThreshold_1QuarterFull ;
DMA_InitStructure.DMA_MemoryBurst       = DMA_MemoryBurst_Single;
DMA_InitStructure.DMA_PeripheralBurst   = DMA_PeripheralBurst_Single;
DMA_Init(DMA2_Stream5, &DMA_InitStructure);
NVIC_EnableIRQ(DMA_Channel_4);
DMA_Cmd(DMA2_Stream5, ENABLE);
```



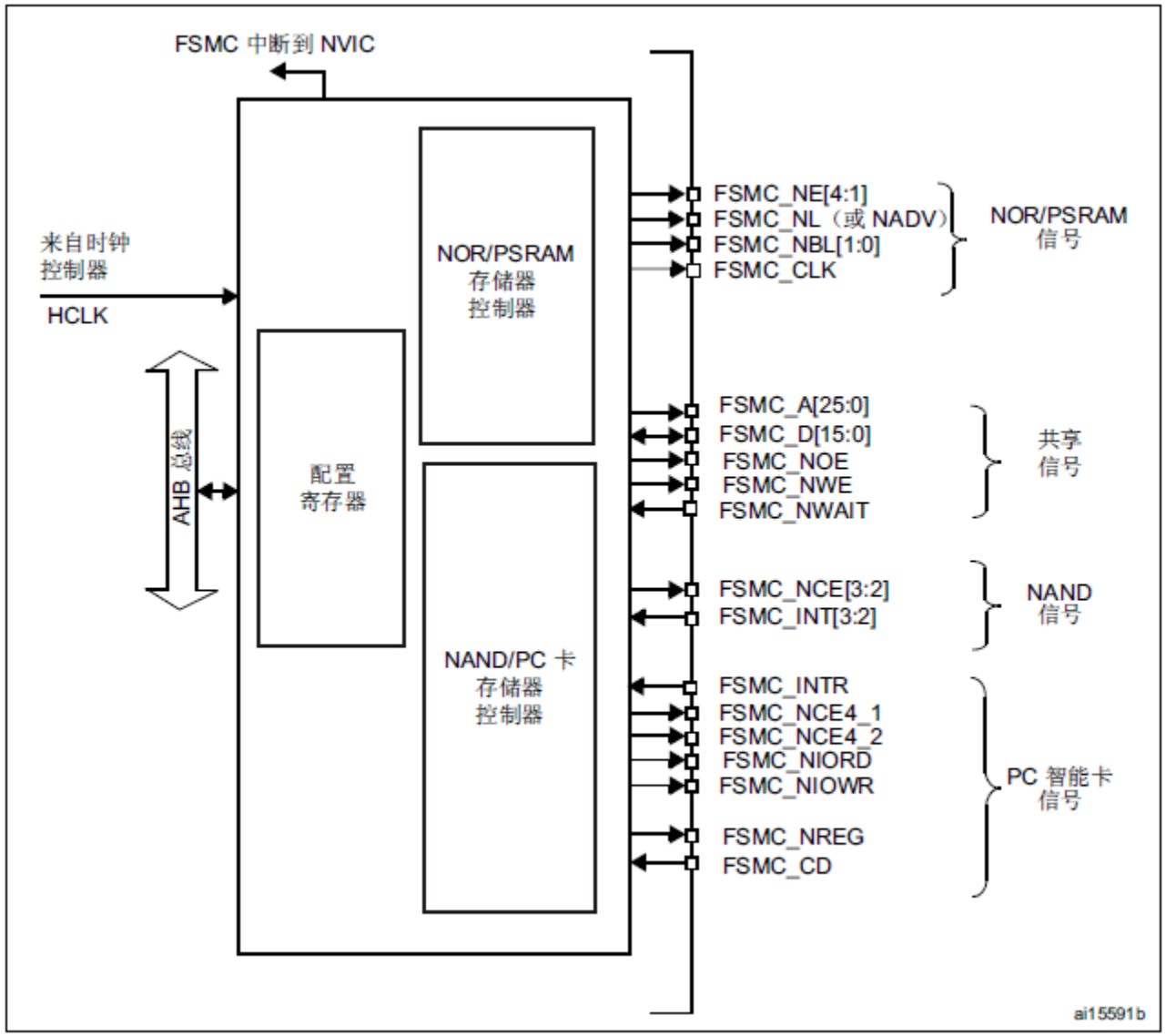
（8）直接内存访问DMA

中断程序：

```
void DMA2_Stream5_IRQHandler(void)
{
        if(DMA_GetITStatus(DMA2_Stream5, DMA_IT_TCIF7))
        {
/* Clear DMA Stream Transfer Complete interrupt pending bit */
                DMA_ClearITPendingBit(DMA2_Stream5, DMA_IT_TCIF7);
                DMA_Cmd(DMA2_Stream5, DISABLE);
                ……
        }
}
```

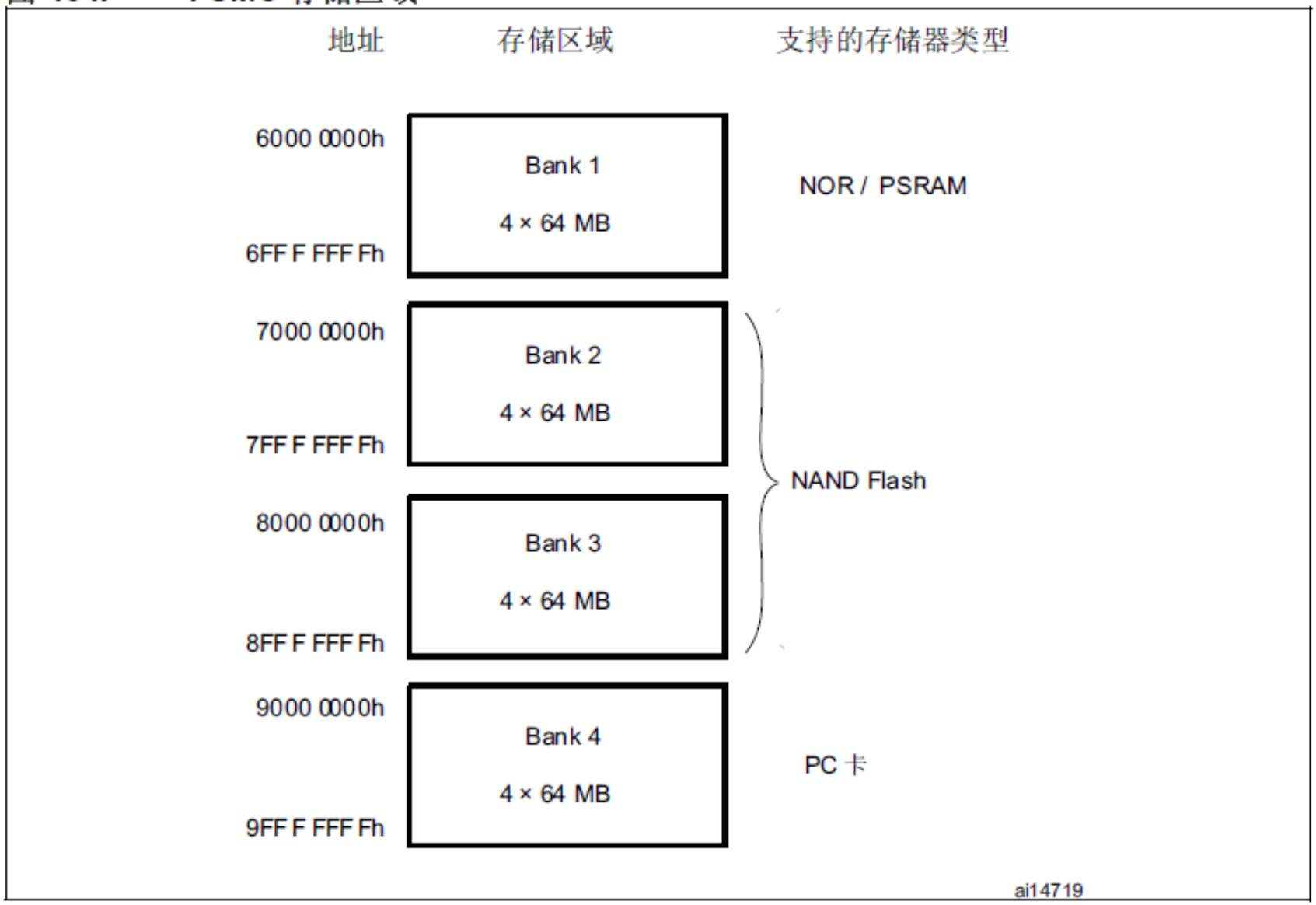
（9）存贮器扩展（FSMC）

FSMC（Flexible static memory controller，灵活的静态存储控制器 ）



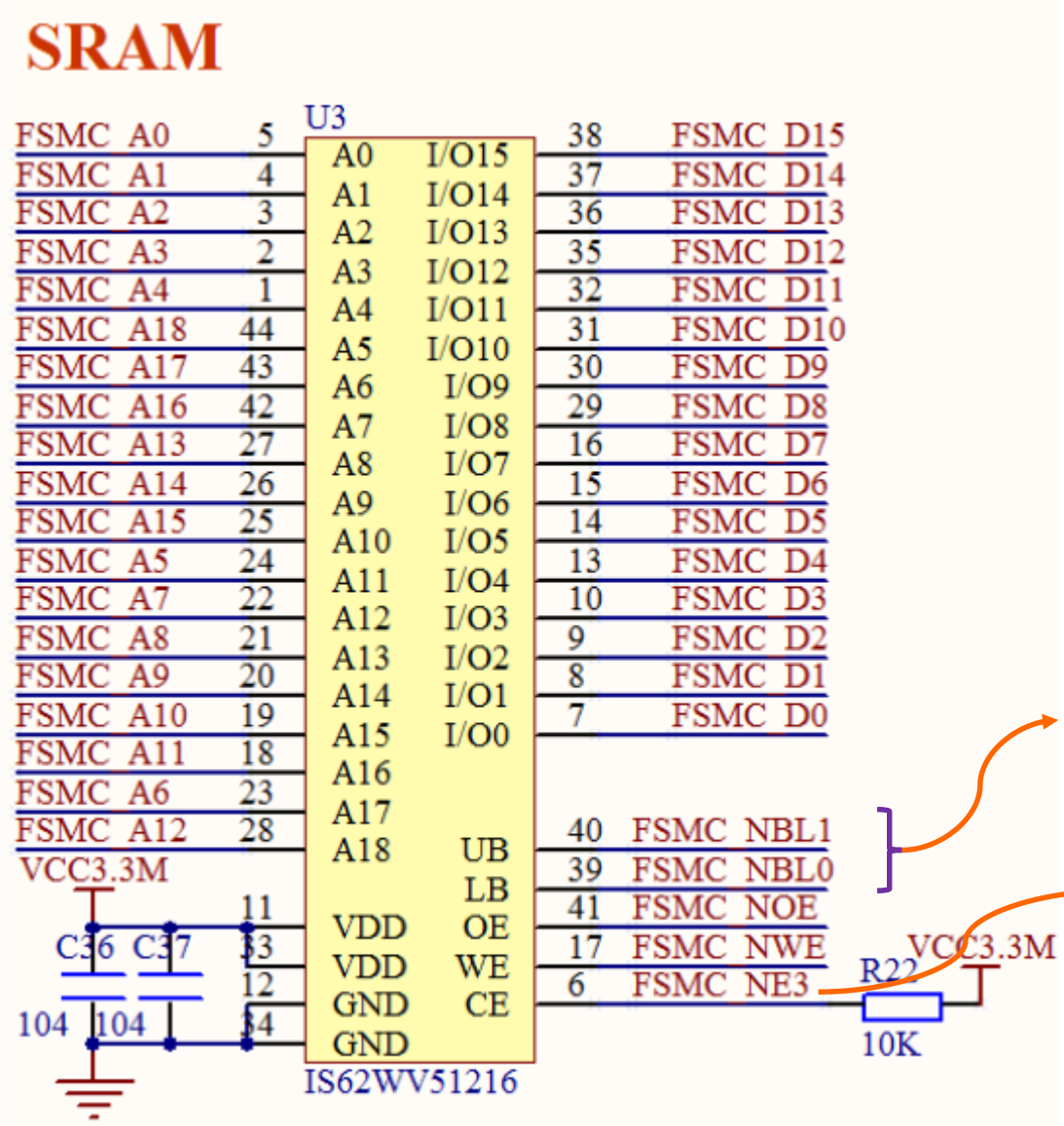
（8）存贮器扩展（FSMC）

图 404.　FSMC 存储区域



（9）存贮器扩展（FSMC）

例1：扩展1M字节SRAM（16位数据，19根地址线）



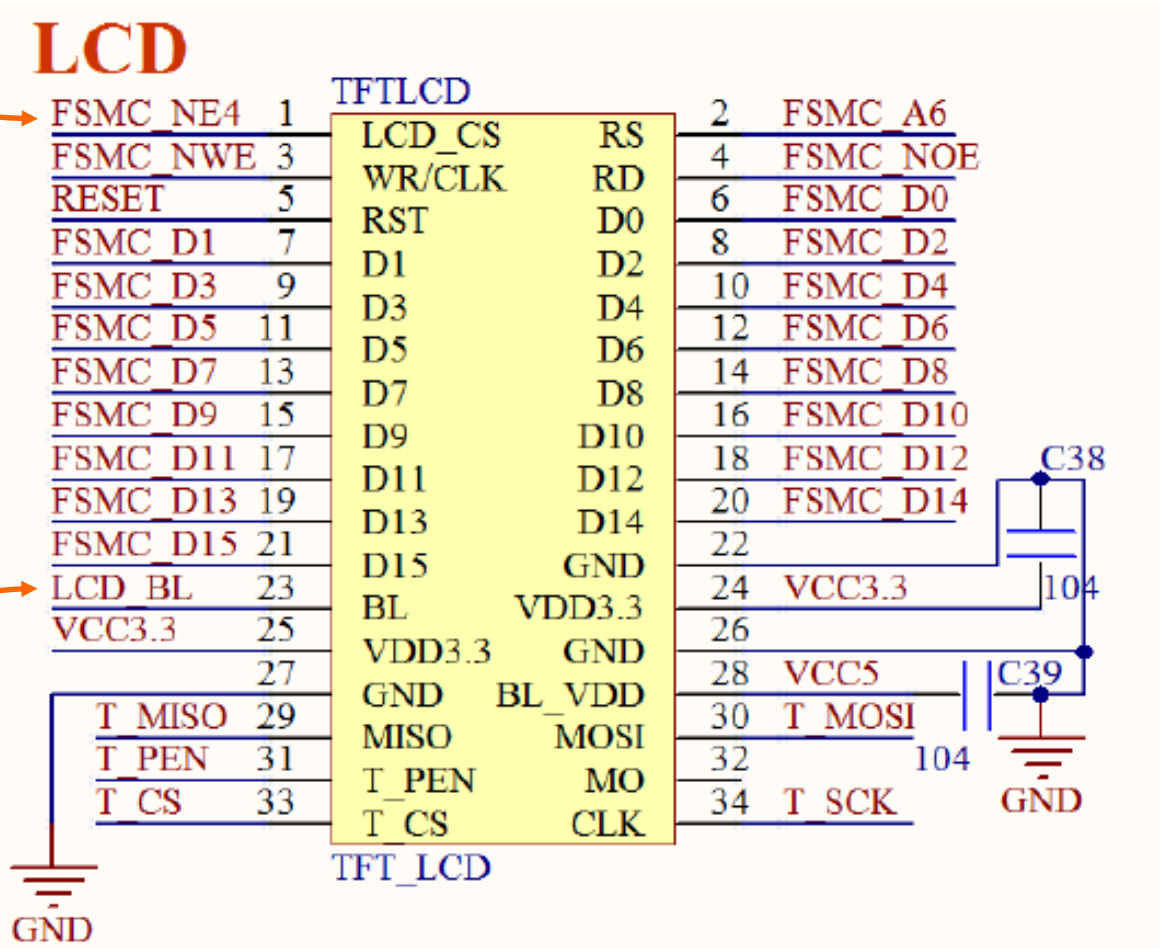
（9）存贮器扩展（FSMC）

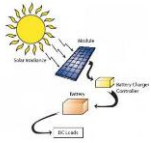
例2：扩展LCD接口

第四扇区，

地址：**0x6c000000**

低字节使能





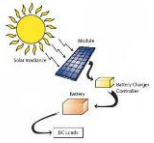
初始化程序：

```
FSMC_NORSRAMInitStructure.FSMC_Bank = FSMC_Bank1_NORSRAM3;//  选择NE3
 FSMC_NORSRAMInitStructure.FSMC_DataAddressMux = FSMC_DataAddressMux_Disable;
 FSMC_NORSRAMInitStructure.FSMC_MemoryType =FSMC_MemoryType_SRAM;//  选择SRAM方式
 FSMC_NORSRAMInitStructure.FSMC_MemoryDataWidth = FSMC_MemoryDataWidth_16b;//存贮器数据宽度16bit
 FSMC_NORSRAMInitStructure.FSMC_BurstAccessMode =FSMC_BurstAccessMode_Disable;
 FSMC_NORSRAMInitStructure.FSMC_WaitSignalPolarity = FSMC_WaitSignalPolarity_Low;
 FSMC_NORSRAMInitStructure.FSMC_AsynchronousWait=FSMC_AsynchronousWait_Disable;
 FSMC_NORSRAMInitStructure.FSMC_WrapMode = FSMC_WrapMode_Disable;
 FSMC_NORSRAMInitStructure.FSMC_WaitSignalActive = FSMC_WaitSignalActive_BeforeWaitState;
 FSMC_NORSRAMInitStructure.FSMC_WriteOperation = FSMC_WriteOperation_Enable;          //存贮器写使能
 FSMC_NORSRAMInitStructure.FSMC_WaitSignal = FSMC_WaitSignal_Disable;
 FSMC_NORSRAMInitStructure.FSMC_ExtendedMode = FSMC_ExtendedMode_Disable; //读写使用相同时序
 FSMC_NORSRAMInitStructure.FSMC_WriteBurst = FSMC_WriteBurst_Disable;
 FSMC_NORSRAMInitStructure.FSMC_ReadWriteTimingStruct = &readWriteTiming;
 FSMC_NORSRAMInitStructure.FSMC_WriteTimingStruct = &readWriteTiming; //读写同样时序
 FSMC_NORSRAMInit(&FSMC_NORSRAMInitStructure);  //初始化FSMC配置
 FSMC_NORSRAMCmd(FSMC_Bank1_NORSRAM3, ENABLE);  // 使能BANK1区域3
```

程序定义RAM：

指定地址

```
uint8 FileBuff[100000]__attribute__((at(0x68000000)));
```



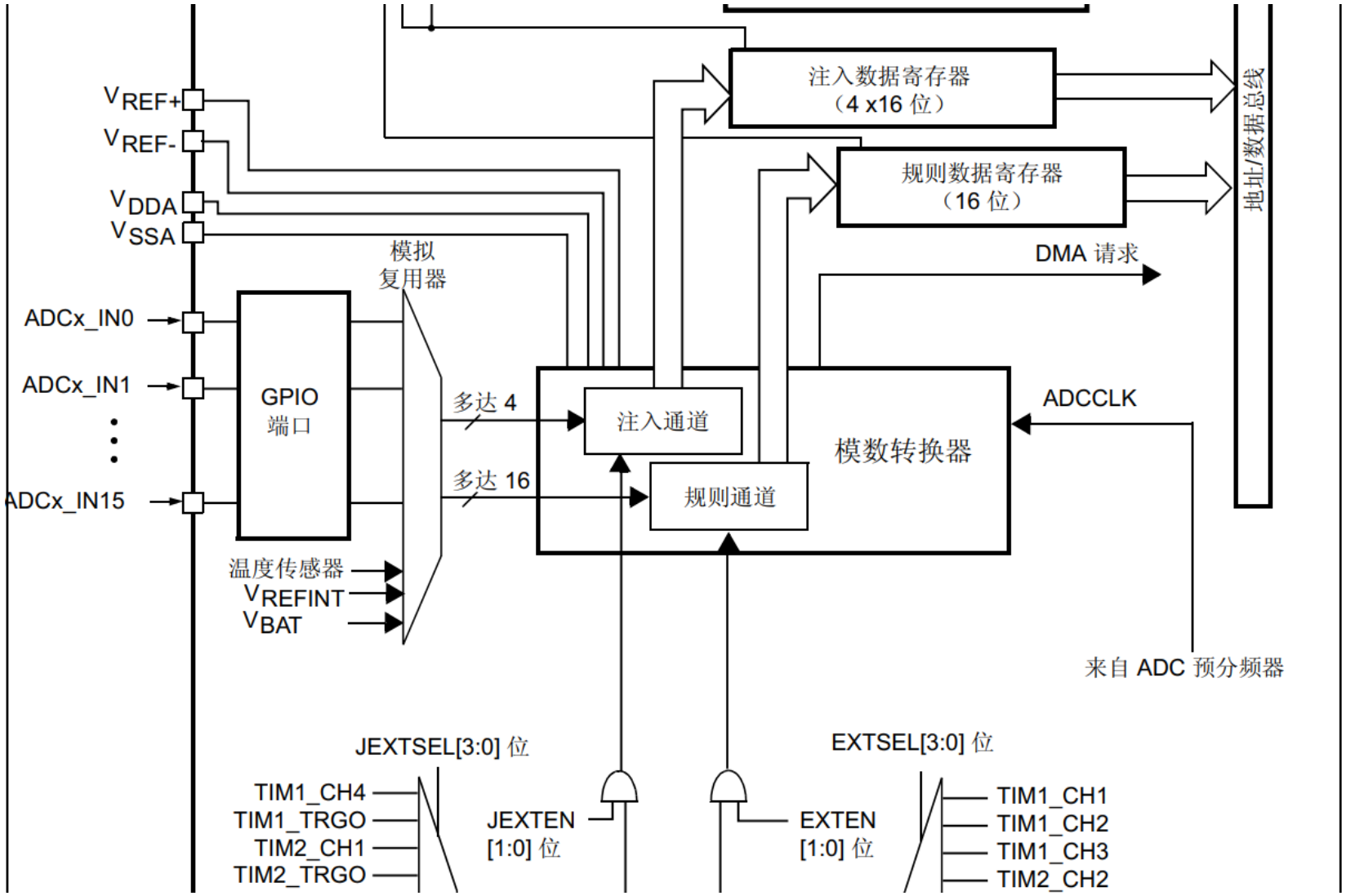
（9）模数转换器（ADC）

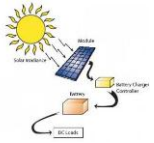
简介： 3个12位逐次趋近型ADC。每个ADC最多19路复用通道，可测量来自16个外部源、两个内部源和VBAT通道的信号。这些通道的A/D转换可在单次、连续、扫描或不连续采样模式下进行。ADC的结果存储在一个左对齐或右对齐的16位数据寄存器中。

ADC 主要特性：

- 可配置 12 位、10 位、8 位或 6 位分辨率
- 在转换结束、注入转换结束以及发生模拟看门狗或溢出事件时产生中断
- 单次和连续转换模式
- 可设置数据左对齐或右对齐
- 可独立设置各通道采样时间
- 多种外部触发器选项

（10）模数转换器（ADC）


注入数据寄存器
（4 x16 位）
规则数据寄存器
（16 位）
地址/数据总线
DMA 请求
VREF+
VREF-
VDDA
VSSA
模拟
复用器
ADCx_IN0
ADCx_IN1
ADCx_IN15
GPIO
端口
多达 4
多达 16
注入通道
规则通道
模数转换器
ADCCLK
温度传感器
VREFINT
VBAT
来自 ADC 预分频器
JEXTSEL[3:0] 位
TIM1_CH4
TIM1_TRGO
TIM2_CH1
TIM2_TRGO
JEXTEN
[1:0] 位
EXTEN
[1:0] 位
EXTSEL[3:0] 位
TIM1_CH1
TIM1_CH2
TIM1_CH3
TIM2_CH2



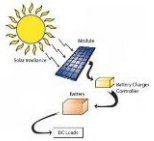
（10）模数转换器（ADC）

ADC 开关控制

- 将 ADC_CR2 寄存器中的 ADON 位置 1 来为 ADC 供电。首次将 ADON 位置 1 时， 会将 ADC 从掉电模式中唤醒。
- SWSTART 或 JSWSTART 位置 1 时，启动 AD 转换。

通道选择

- 有 16 条复用通道。可以将转换分为两组：规则转换和注入转换。每个组包含一个转换序列， 该序列可按任意顺序在任意通道上完成。例如，可按以下顺序对序列进行转换：ADC_IN3、ADC_IN8、ADC_IN2、ADC_IN2、ADC_IN0、ADC_IN2、ADC_IN2、ADC_IN15。
- 一个规则转换组最多由 16 个转换构成。必须在 ADC_SQRx 寄存器中选择转换序列的规则通道及其顺序。



（10）模数转换器（ADC）

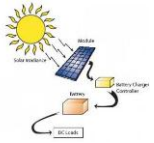
单次转换模式

在单次转换模式下，ADC 执行一次转换。CONT 位为 0 时，可通过以下方式启动此模式：

- 将 ADC_CR2 寄存器的 SWSTART 位置 1（用于规则通道）
- 将 JSWSTART 位置 1（适用于注入通道）
- 外部触发（适用于规则通道或注入通道）

连续转换模式

在连续转换模式下，ADC 结束一个转换后立即启动一个新的转换。CONT 位为 1 时，可通过外部触发或将 ADC_CR2 寄存器的 SWSTRT 位置 1 来启动此模式（仅适用于规则通道）。



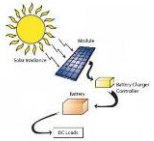
（10）模数转换器（ADC）

启动ADC转换的外部触发事件

- 定时器事件
- EXTI 中断

表 51.　规则通道的外部触发

源	类型	EXTSEL[3:0]
TIM1_CH1 事件	片上定时器的内部信号	0000
TIM1_CH2 事件		0001
TIM1_CH3 事件		0010
TIM2_CH2 事件		0011
TIM2_CH3 事件		0100
TIM2_CH4 事件		0101
TIM2_TRGO 事件		0110
TIM3_CH1 事件		0111
TIM3_TRGO 事件		1000
TIM4_CH4 事件		1001
TIM5_CH1 事件		1010
TIM5_CH2 事件		1011
TIM5_CH3 事件		1100
TIM8_CH1 事件		1101
TIM8_TRGO 事件		1110
EXTI 线 11	外部引脚	1111



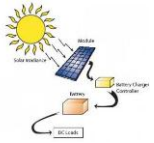
（10）模数转换器（ADC）

AD数据的处理

1、直接读ADC_DR 寄存器（使用中断）

2、使用 **DMA**

- 使用 DMA 可以避免丢失在下一次AD转换前还未被读出的 ADC_DR 寄存器中的数据。
- 在使能 DMA 模式的情况下（ADC_CR2 寄存器中的 DMA 位置 1），每完成规则通道组中的 一个通道转换后，都会生成一个 DMA 请求。这样便可将转换的数据从 ADC_DR 寄存器传输 到用软件选择的目标位置。



（10）模数转换器（ADC）

例子：以1Msps的采样率，从GPIOA.1采样信号，保存到AdcDataBuffer数组，初始化程序。

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2, ENABLE);
RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
```

开启用到的外设

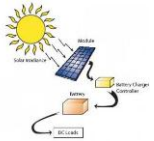
```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AN;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL ;
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

对GPIOA.1初始化，选模拟量方式

```
DMA_InitStructure.DMA_Channel = DMA_Channel_0;
DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)&ADC1->DR;
DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)(&AdcDataBuffer);
DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
DMA_InitStructure.DMA_BufferSize = NUM_OF_ADC;
DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
…

DMA_Init(DMA2_Stream0, &DMA_InitStructure);
```

设置DMA的源地址和目标地址



（10）模数转换器（ADC）

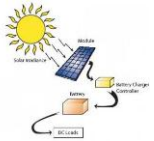
ADC初始化

```
 ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;
 ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div4;
 ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled;
ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_5Cycles;
ADC_CommonInit(&ADC_CommonInitStructure);

/* ADC1 regular channel 1 configuration ************************************/
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode = DISABLE;
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_Rising;
ADC_InitStructure.ADC_ExternalTrigConv=ADC_ExternalTrigConv_T8_TRGO;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfConversion = 1;
ADC_Init(ADC1, &ADC_InitStructure);

/* Enable ADC1 DMA */
 ADC_DMACmd(ADC1, ENABLE);
```

设置ADC的触发源



（10）模数转换器（ADC）

TIM8初始化

```
void Ini_TIM8(void)
{
 TIM_OCInitTypeDef  TIM_OCInitStructure;
 TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
 RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM8 , ENABLE);
 TIM_DeInit(TIM8);
 TIM_TimeBaseStructure.TIM_Period = 167;
 TIM_TimeBaseStructure.TIM_Prescaler= 0;
 TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;
 TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;
 TIM_TimeBaseInit(TIM8, &TIM_TimeBaseStructure);
 TIM_ClearFlag(TIM8, TIM_FLAG_CC1);
}
```

设置TIM8定时时间为1us

旋转显示屏设计（讨论）

1、电路与结构

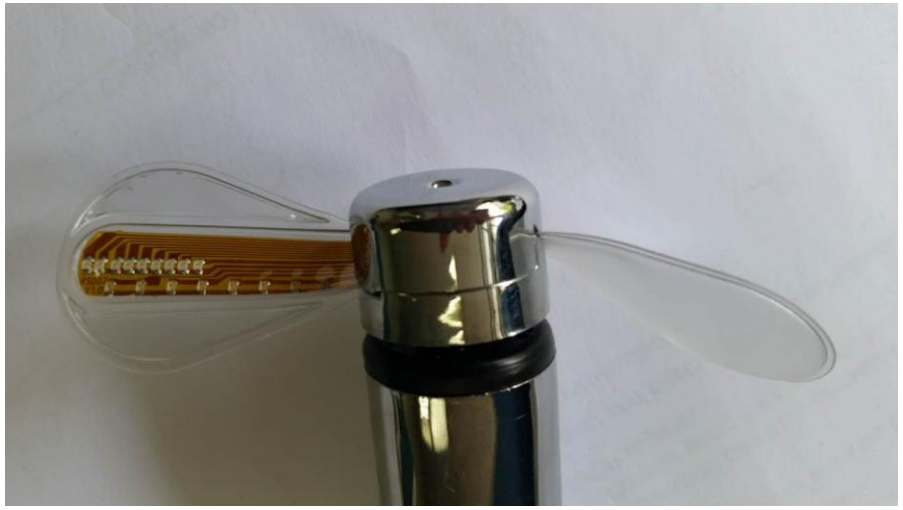
1、如何解决旋转部分的供电问题；
2、单片机最好不要跟随风扇旋转，电路如何设计。

2、风扇叶片定位

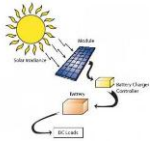
如何确定当前转速，从而使显示与旋转保持同步。

3、程序架构设计

1、程序架构如何设计，包括哪些部分？
2、各部分程序如何编写？







END

THANK YOU!